CAPÍTULO 3

Descripción y control de procesos

3.1. ¿Qué es un proceso?

3.2. Estados de los procesos

3.3. Descripción de los procesos

3.4. Control de procesos

3.5. Gestión de procesos en UNIX SVR4

3.6. Resumen

3.7. Lecturas recomendadas

3.8. Términos clave, cuestiones de repaso, y problemas

El diseño de un sistema operativo debe reflejar ciertos requisitos generales. Todos los sistemas operativos multiprogramados, desde los sistemas operativos monousuario como Windows 98 hasta sistemas *mainframes* como IBM z/OS, que son capaces de dar soporte a miles de usuarios, se construyen en torno al concepto de proceso. La mayoría de los requisitos que un sistema operativo debe cumplir se pueden expresar con referencia a los procesos:

- El sistema operativo debe intercalar la ejecución de múltiples procesos, para maximizar la utilización del procesador mientras se proporciona un tiempo de respuesta razonable.
- El sistema operativo debe reservar recursos para los procesos conforme a una política específica (por ejemplo, ciertas funciones o aplicaciones son de mayor prioridad) mientras que al mismo tiempo evita interbloqueos[1].
- Un sistema operativo puede requerir dar soporte a la comunicación entre procesos y la creación de procesos, mediante las cuales ayuda a la estructuración de las aplicaciones.

Se comienza el estudio detallado de los sistemas operativos examinando la forma en la que éstos representan y controlan los procesos. Después de una introducción al concepto de proceso, el capítulo presentará los estados de los procesos, que caracterizan el comportamiento de los mismos. Seguidamente, se presentarán las estructuras de datos que el sistema operativo usa para gestionar los procesos. Éstas incluyen las estructuras para representar el estado de cada proceso así como para registrar características de los mismos que el sistema operativo necesita para alcanzar sus objetivos. Posteriormente, se verá cómo el sistema operativo utiliza estas estructuras para controlar la ejecución de los procesos. Por último, se discute la gestión de procesos en UNIX SVR4. El Capítulo 4 proporciona ejemplos más modernos de gestión de procesos, tales como Solaris, Windows, y Linux.

Nota: en este capítulo hay referencias puntuales a la memoria virtual. La mayoría de las veces podemos ignorar este concepto en relación con los procesos, pero en ciertos puntos de la discusión, las consideraciones sobre memoria virtual se hacen pertinentes. La memoria virtual no se discutirá en detalle hasta el Capítulo 8; ya se ha proporcionado una somera visión general en el Capítulo 2.

3.1. ¿QUÉ ES UN PROCESO?

CONCEPTOS PREVIOS

Antes de definir el término proceso, es útil recapitular algunos de los conceptos ya presentados en los Capítulos 1 y 2:

1. Una plataforma de computación consiste en una colección de recursos hardware, como procesador, memoria, módulos de E/S, relojes, unidades de disco y similares.
2. Las aplicaciones para computadores se desarrollan para realizar determinadas tareas. Suelen aceptar entradas del mundo exterior, realizar algún procesamiento y generar salidas.
3. No es eficiente que las aplicaciones estén escritas directamente para una plataforma hardware específica. Las principales razones son las siguientes:

---

[1] Los interbloqueos se examinarán en el Capítulo 6. Como ejemplo sencillo, un interbloqueo ocurre si dos procesos necesitan dos recursos iguales para continuar y cada uno de los procesos tiene la posesión de uno de los recursos. A menos que se realice alguna acción, cada proceso esperará indefinidamente por conseguir el otro recurso.

a) Numerosas aplicaciones pueden desarrollarse para la misma plataforma, de forma que tiene sentido desarrollar rutinas comunes para acceder a los recursos del computador.

b) El procesador por sí mismo proporciona únicamente soporte muy limitado para la multiprogramación. Es necesario disponer de software para gestionar la compartición del procesador así como otros recursos por parte de múltiples aplicaciones al mismo tiempo.

c) Cuando múltiples aplicaciones están activas al mismo tiempo es necesario proteger los datos, el uso de la E/S y los recursos propios de cada aplicación con respecto a las demás.

4. El sistema operativo se desarrolló para proporcionar una interfaz apropiada para las aplicaciones, rica en funcionalidades, segura y consistente. El sistema operativo es una capa de software entre las aplicaciones y el hardware del computador (Figura 2.1) que da soporte a aplicaciones y utilidades.
5. Se puede considerar que el sistema operativo proporciona una representación uniforme y abstracta de los recursos, que las aplicaciones pueden solicitar y acceder. Los recursos incluyen la memoria principal, las interfaces de red, los sistemas de ficheros, etc. Una vez que el sistema operativo ha creado estas abstracciones de los recursos para que las aplicaciones las usen, debe también controlar su uso. Por ejemplo, un sistema operativo podría permitir compartición y protección de recursos.

Ahora que se conocen los conceptos de aplicaciones, software de sistema y de recursos se está en disposición de hablar sobre cómo un sistema operativo puede, de forma ordenada, gestionar la ejecución de aplicaciones de forma que:

- Los recursos estén disponibles para múltiples aplicaciones.
- El procesador físico se conmute entre múltiples aplicaciones, de forma que todas lleguen a procesarse.
- El procesador y los dispositivos de E/S se puedan usar de forma eficiente.

El enfoque adoptado por todos los sistemas operativos modernos recae en un modelo bajo el cual la ejecución de una aplicación se corresponde con la existencia de uno o más procesos.

PROCESOS Y BLOQUES DE CONTROL DE PROCESOS

Se debe recordar que en el Capítulo 2 se sugirieron diversas definiciones del término *proceso*, incluyendo:

- Un programa en ejecución.
- Una instancia de un programa ejecutado en un computador.
- La entidad que se puede asignar y ejecutar en un procesador.
- Una unidad de actividad que se caracteriza por la ejecución de una secuencia de instrucciones, un estado actual, y un conjunto de recursos del sistema asociados.

También se puede pensar en un proceso como en una entidad que consiste en un número de elementos. Los dos elementos esenciales serían el **código de programa** (que puede compartirse con otros procesos que estén ejecutando el mismo programa) y un **conjunto de datos** asociados a dicho

código. Supongamos que el procesador comienza a ejecutar este código de programa, y que nos referiremos a esta entidad en ejecución como un proceso. En cualquier instante puntual del tiempo, *mientras el proceso está en ejecución*, este proceso se puede caracterizar por una serie de elementos, incluyendo los siguientes:

- **Identificador.** Un identificador único asociado a este proceso, para distinguirlo del resto de procesos.
- **Estado.** Si el proceso está actualmente corriendo, está en el estado *en ejecución*.
- **Prioridad**: Nivel de prioridad relativo al resto de procesos.
- **Contador de programa.** La dirección de la siguiente instrucción del programa que se ejecutará.
- **Punteros a memoria.** Incluye los punteros al código de programa y los datos asociados a dicho proceso, además de cualquier bloque de memoria compartido con otros procesos.
- **Datos de contexto.** Estos son datos que están presenten en los registros del procesador cuando el proceso está corriendo.
- **Información de estado de E/S.** Incluye las peticiones de E/S pendientes, dispositivos de E/S (por ejemplo, una unidad de cinta) asignados a dicho proceso, una lista de los ficheros en uso por el mismo, etc.
- **Información de auditoría.** Puede incluir la cantidad de tiempo de procesador y de tiempo de reloj utilizados, así como los límites de tiempo, registros contables, etc.

La información de la lista anterior se almacena en una estructura de datos, que se suele llamar bloque de control de proceso (*process control block*) (Figura 3.1), que el sistema operativo crea y gestiona. El punto más significativo en relación al bloque de control de proceso, o BCP, es que contiene suficiente información de forma que es posible interrumpir el proceso cuando está corriendo y posteriormente restaurar su estado de ejecución como si no hubiera habido interrupción alguna. El BCP es la herramienta clave que permite al sistema operativo dar soporte a múltiples procesos y proporcionar multiprogramación. Cuando un proceso se interrumpe, los valores actuales del contador de programa y los registros del procesador (datos de contexto) se guardan en los campos correspondientes del BCP y el estado del proceso se cambia a cualquier otro valor, como *bloqueado* o *listo* (descritos a continuación). El sistema operativo es libre ahora para poner otro proceso en estado de ejecución. El contador de programa y los datos de contexto se recuperan y cargan en los registros del procesador y este proceso comienza a correr.

De esta forma, se puede decir que un proceso está compuesto del código de programa y los datos asociados, además del bloque de control de proceso o BCP. Para un computador monoprocesador, en un instante determinado, como máximo un único proceso puede estar corriendo y dicho proceso estará en el estado *en ejecución*.

3.2. ESTADOS DE LOS PROCESOS

Como se acaba de comentar, para que un programa se ejecute, se debe crear un proceso o tarea para dicho programa. Desde el punto de vista del procesador, él ejecuta instrucciones de su repertorio de instrucciones en una secuencia dictada por el cambio de los valores del registro contador de programa. A lo largo del tiempo, el contador de programa puede apuntar al código de diferentes programas que son parte de diferentes procesos. Desde el punto de vista de un programa individual, su ejecución implica una secuencia de instrucciones dentro de dicho programa.

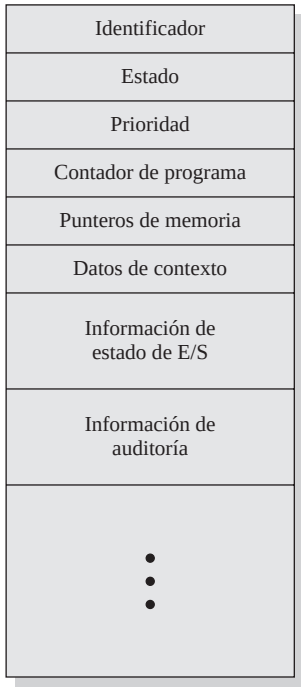


Figura 3.1. Bloque de control de programa (BCP) simplificado.

Se puede caracterizar el comportamiento de un determinado proceso, listando la secuencia de instrucciones que se ejecutan para dicho proceso. A esta lista se la denomina **traza** del proceso. Se puede caracterizar el comportamiento de un procesador mostrando cómo las trazas de varios procesos se entrelazan.

Considere un ejemplo. La Figura 3.2 muestra el despliegue en memoria de tres procesos. Para simplificar la exposición, se asume que dichos procesos no usan memoria virtual; por tanto, los tres procesos están representados por programas que residen en memoria principal. De manera adicional, existe un pequeño programa **activador** (*dispatcher*) que intercambia el procesador de un proceso a otro. La Figura 3.3 muestra las trazas de cada uno de los procesos en los primeros instantes de ejecución. Se muestran las 12 primeras instrucciones ejecutadas por los procesos A y C. El proceso B ejecuta 4 instrucciones y se asume que la cuarta instrucción invoca una operación de E/S, a la cual el proceso debe esperar.

Ahora vea estas trazas desde el punto de vista del procesador. La Figura 3.4 muestra las trazas entrelazadas resultante de los 52 primeros ciclos de ejecución (por conveniencia los ciclos de instrucciones han sido numerados). En este ejemplo, se asume que el sistema operativo sólo deja que un proceso continúe durante seis ciclos de instrucción, después de los cuales se interrumpe; lo cual previene que un solo proceso monopolice el uso del tiempo del procesador. Como muestra la Figura 3.4, las primeras seis instrucciones del proceso A se ejecutan seguidas de una alarma de temporización (*time-out*) y de la ejecución de cierto código del activador, que ejecuta seis instrucciones antes de devolver el control al proceso B[2]. Después de que se ejecuten cuatro instrucciones, el proceso B solicita una acción de

[2] El reducido número de instrucciones ejecutadas por los procesos y por el planificador es irreal; se ha utilizado para simplificar el ejemplo y clarificar las explicaciones.

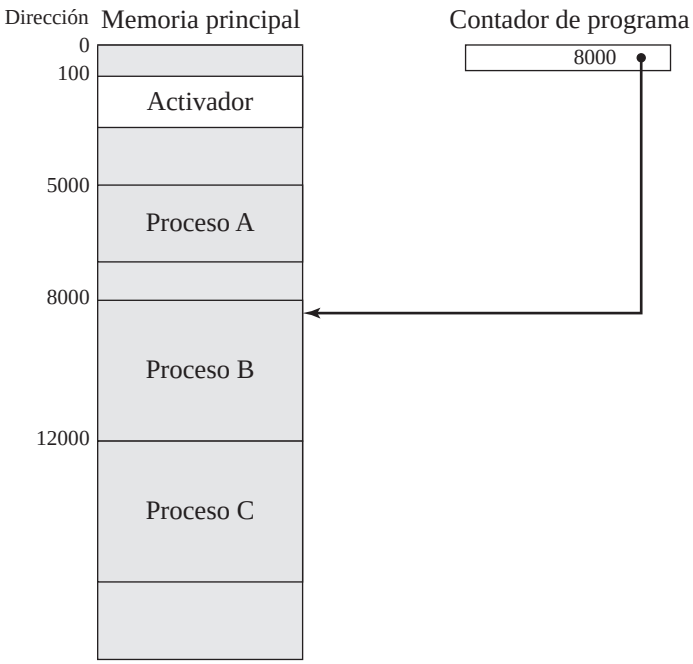


Figura 3.2. Instantánea de un ejemplo de ejecución (Figura 3.4) en el ciclo de instrucción 13.

5000	8000	12000
5001	8001	12001
5002	8002	12002
5003	8003	12003
5004		12004
5005		12005
5006		12006
5007		12007
5008		12008
5009		12009
5010		12010
5011		12011
(a) Traza del Proceso A	**(b) Traza del Proceso B**	**(c) Traza del Proceso C**

5000 = Dirección de comienzo del programa del Proceso A.
8000 = Dirección de comienzo del programa del Proceso B.
12000 = Dirección de comienzo del programa del Proceso C.

Figura 3.3. Traza de los procesos de la Figura 3.2.

E/S, para la cual debe esperar. Por tanto, el procesador deja de ejecutar el proceso B y pasa a ejecutar el proceso C, por medio del activador. Después de otra alarma de temporización, el procesador vuelve al proceso A. Cuando este proceso llega a su temporización, el proceso B aún estará esperando que se complete su operación de E/S, por lo que el activador pasa de nuevo al proceso C.

UN MODELO DE PROCESO DE DOS ESTADOS

La responsabilidad principal del sistema operativo es controlar la ejecución de los procesos; esto incluye determinar el patrón de entrelazado para la ejecución y asignar recursos a los procesos. El primer paso en el diseño de un sistema operativo para el control de procesos es describir el comportamiento que se desea que tengan los procesos.

Se puede construir el modelo más simple posible observando que, en un instante dado, un proceso está siendo ejecutando por el procesador o no. En este modelo, un proceso puede estar en dos estados: Ejecutando o No Ejecutando, como se muestra en la Figura 3.5a. Cuando el sistema operativo crea un nuevo proceso, crea el bloque de control de proceso (BCP) para el nuevo proceso e inserta dicho proceso en el sistema en estado No Ejecutando. El proceso existe, es conocido por el sistema operativo, y está esperando su oportunidad de ejecutar. De cuando en cuando, el proceso actualmente en ejecución se interrumpirá y una parte del sistema operativo, el activador, seleccionará otro proceso

1	5000	
2	5001	
3	5002	
4	5003	
5	5004	
6	5005	
		Temporización
7	100	
8	101	
9	102	
10	103	
11	104	
12	105	
13	8000	
14	8001	
15	8002	
16	8003	
		Petición de E/S
17	100	
18	101	
19	102	
20	103	
21	104	
22	105	
23	12000	
24	12001	
25	12002	
26	12003	
27	12004	
28	12005	
		Temporización
29	100	
30	101	
31	102	
32	103	
33	104	
34	105	
35	5006	
36	5007	
37	5008	
38	5009	
39	5010	
40	5011	
		Temporización
41	100	
42	101	
43	102	
44	103	
45	104	
46	105	
47	12006	
48	12007	
49	12008	
50	12009	
51	12010	
52	12011	
		Temporización

100 = Dirección de comienzo del programa activador.
Las zonas sombreadas indican la ejecución del proceso de activación;
la primera y la tercera columna cuentan ciclos de instrucciones;
la segunda y la cuarta columna las direcciones de las instrucciones que se ejecutan

Figura 3.4. Traza combinada de los procesos de la Figura 3.2.

a ejecutar. El proceso saliente pasará del estado Ejecutando a No Ejecutando y pasará a Ejecutando un nuevo proceso.

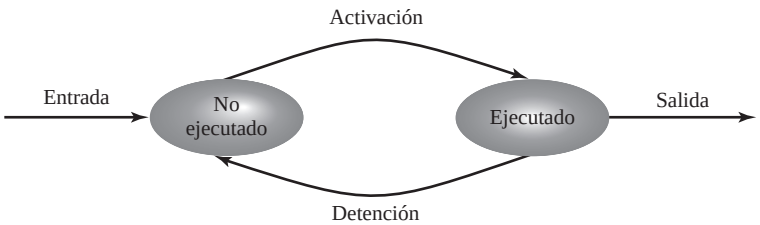
De este modelo simple, ya se puede apreciar algo del diseño de los elementos del sistema operativo. Cada proceso debe representarse de tal manera que el sistema operativo pueda seguirle la pista. Es decir, debe haber información correspondiente a cada proceso, incluyendo el estado actual y su localización en memoria; esto es el bloque de control de programa. Los procesos que no están ejecutando deben estar en una especie de cola, esperando su turno de ejecución. La Figura 3.5b sugiere esta estructura. Existe una sola cola cuyas entradas son punteros al BCP de un proceso en particular. Alternativamente, la cola debe consistir en una lista enlazada de bloques de datos, en la cual cada bloque representa un proceso; exploraremos posteriormente esta última implementación.

Podemos describir el comportamiento del activador en términos de este diagrama de colas. Un proceso que se interrumpe se transfiere a la cola de procesos en espera. Alternativamente, si el proceso ha finalizado o ha sido abortado, se descarta (sale del sistema). En cualquier caso, el activador selecciona un proceso de la cola para ejecutar.

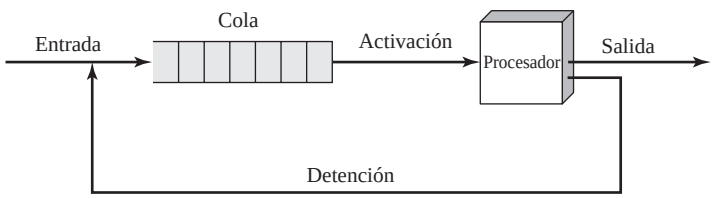
CREACIÓN Y TERMINACIÓN DE PROCESOS

Antes de intentar refinar nuestro sencillo modelo de dos estados, resultará útil hablar de la creación y terminación de los procesos; por último, y de forma independiente del modelo de comportamiento de procesos que se use, la vida de un proceso está acotada entre su creación y su terminación.

Creación de un proceso. Cuando se va a añadir un nuevo proceso a aquellos que se están gestionando en un determinado momento, el sistema operativo construye las estructuras de datos que se usan para manejar el proceso (como se describió en la Sección 3.3) y reserva el espacio de direcciones en memoria principal para el proceso. Estas acciones constituyen la creación de un nuevo proceso.



(a) Diagrama de transiciones de estados



(b) Modelos de colas

Figura 3.5. Modelo de proceso de dos estados.

Existen cuatro eventos comunes que llevan a la creación de un proceso, como se indica en la Tabla 3.1. En un entorno por lotes, un proceso se crea como respuesta a una solicitud de trabajo. En un entorno interactivo, un proceso se crea cuando un nuevo usuario entra en el sistema. En ambos casos el sistema operativo es responsable de la creación de nuevos procesos. Un sistema operativo puede, a petición de una aplicación, crear procesos. Por ejemplo, si un usuario solicita que se imprima un fichero, el sistema operativo puede crear un proceso que gestione la impresión. El proceso solicitado puede, de esta manera, operar independientemente del tiempo requerido para completar la tarea de impresión.

TABLA 3.1. Razones para la creación de un proceso.

Nuevo proceso de lotes	El sistema operativo dispone de un flujo de control de lotes de trabajos, habitualmente una cinta un disco. Cuando el sistema operativo está listo para procesar un nuevo trabajo, leerá la siguiente secuencia de mandatos de control de trabajos.
Sesión interactiva	Un usuario desde un terminal entra en el sistema.
Creado por el sistema operativo para proporcionar un servicio	El sistema operativo puede crear un proceso para realizar una función en representación de un programa de usuario, sin que el usuario tenga que esperar (por ejemplo, un proceso para controlar la impresión).
Creado por un proceso existente	Por motivos de modularidad o para explotar el paralelismo, un programa de usuario puede ordenar la creación de un número de procesos.

Tradicionalmente, un sistema operativo creaba todos los procesos de una forma que era transparente para usuarios y programas. Esto aún es bastante común en muchos sistemas operativos contemporáneos. Sin embargo, puede ser muy útil permitir a un proceso la creación de otro. Por ejemplo, un proceso de aplicación puede generar otro proceso para recibir los datos que la aplicación está generando y para organizar esos datos de una forma apropiada para su posterior análisis. El nuevo proceso ejecuta en paralelo con el proceso original y se activa cuando los nuevos datos están disponibles. Esta organización puede ser muy útil en la estructuración de la aplicación. Otro ejemplo, un proceso servidor (por ejemplo, un servidor de impresoras, servidor de ficheros) puede generar un proceso por cada solicitud que esté manejando. Cuando un sistema operativo crea un proceso a petición explícita de otro proceso, dicha acción se denomina ***creación del proceso***.

Cuando un proceso lanza otro, al primero se le denomina **proceso padre**, y al proceso creado se le denomina **proceso hijo**. Habitualmente, la relación entre procesos necesita comunicación y cooperación entre ellos. Alcanzar esta cooperación es una tarea complicada para un programador; este aspecto se verá en el Capítulo 5.

Terminación de procesos. La Tabla 3.2 resume las razones típicas para la terminación de un proceso. Todo sistema debe proporcionar los mecanismos mediante los cuales un proceso indica su finalización, o que ha completado su tarea. Un trabajo por lotes debe incluir una instrucción HALT o una llamada a un servicio de sistema operativo especifica para su terminación. En el caso anterior, la instrucción HALT generará una interrupción para indicar al sistema operativo que dicho proceso ha finalizado. Para una aplicación interactiva, las acciones del usuario indicarán cuando el proceso ha terminado. Por ejemplo, en un sistema de tiempo compartido, el proceso de un usuario en particular puede terminar cuando el usuario sale del sistema o apaga su terminal. En un ordenador personal o una estación de trabajo, el usuario puede salir de una aplicación (por ejemplo, un procesador de texto o una

hoja de cálculo). Todas estas acciones tienen como resultado final la solicitud de un servicio al sistema operativo para terminar con el proceso solicitante.

Adicionalmente, un número de error o una condición de fallo puede llevar a la finalización de un proceso. La Tabla 3.2 muestra una lista de las condiciones más habituales de finalización por esos motivos[3].

Tabla 3.2. Razones para la terminación de un proceso.

Finalización normal	El proceso ejecuta una llamada al sistema operativo para indicar que ha completado su ejecución
Límite de tiempo excedido	El proceso ha ejecutado más tiempo del especificado en un límite máximo. Existen varias posibilidades para medir dicho tiempo. Estas incluyen el tiempo total utilizado, el tiempo utilizado únicamente en ejecución, y, en el caso de procesos interactivos, la cantidad de tiempo desde que el usuario realizó la última entrada.
Memoria no disponible	El proceso requiere más memoria de la que el sistema puede proporcionar.
Violaciones de frontera	El proceso trata de acceder a una posición de memoria a la cual no tiene acceso permitido.
Error de protección	El proceso trata de usar un recurso, por ejemplo un fichero, al que no tiene permitido acceder, o trata de utilizarlo de una forma no apropia, por ejemplo, escribiendo en un fichero de sólo lectura.
Error aritmético	El proceso trata de realizar una operación de cálculo no permitida, tal como una división por 0, o trata de almacenar números mayores de los que la representación hardware puede codificar.
Límite de tiempo	El proceso ha esperado más tiempo que el especificado en un valor máximo para que se cumpla un determinado evento.
Fallo de E/S	Se ha producido un error durante una operación de entrada o salida, por ejemplo la imposibilidad de encontrar un fichero, fallo en la lectura o escritura después de un límite máximo de intentos (cuando, por ejemplo, se encuentra un área defectuosa en una cinta), o una operación inválida (la lectura de una impresora en línea).
Instrucción no válida	El proceso intenta ejecutar una instrucción inexistente (habitualmente el resultado de un salto a un área de datos y el intento de ejecutar dichos datos).
Instrucción privilegiada	El proceso intenta utilizar una instrucción reservada al sistema operativo.
Uso inapropiado de datos	Una porción de datos es de tipo erróneo o no se encuentra inicializada.
Intervención del operador por el sistema operativo	Por alguna razón, el operador o el sistema operativo ha finalizado el proceso (por ejemplo, se ha dado una condición de interbloqueo).
Terminación del proceso padre	Cuando un proceso padre termina, el sistema operativo puede automáticamente finalizar todos los procesos hijos descendientes de dicho padre.
Solicitud del proceso padre	Un proceso padre habitualmente tiene autoridad para finalizar sus propios procesos descendientes.

[3] Un sistema operativo compasivo podría en algunos casos permitir al usuario recuperarse de un fallo sin terminar el proceso. Por ejemplo, si un usuario solicita acceder a un fichero y se deniega ese acceso, el sistema operativo podría simplemente informar al usuario de ese hecho y permitir que el proceso continúe.

Por último, en ciertos sistemas operativos, un proceso puede terminarse por parte del proceso que lo creó o cuando dicho proceso padre a su vez ha terminado.

MODELO DE PROCESO DE CINCO ESTADOS

Si todos los procesos estuviesen siempre preparados para ejecutar, la gestión de colas proporcionada en la Figura 3.5b sería efectiva. La cola es una lista de tipo FIFO y el procesador opera siguiendo una estrategia cíclica (*round-robin* o turno rotatorio) sobre todos los procesos disponibles (cada proceso de la cola tiene cierta cantidad de tiempo, por turnos, para ejecutar y regresar de nuevo a la cola, a menos que se bloquee). Sin embargo, hasta con el sencillo ejemplo que vimos antes, esta implementación es inadecuada: algunos procesos que están en el estado de No Ejecutando están listos para ejecutar, mientras que otros están bloqueados, esperando a que se complete una operación de E/S. Por tanto, utilizando una única cola, el activador no puede seleccionar únicamente los procesos que lleven más tiempo en la cola. En su lugar, debería recorrer la lista buscando los procesos que no estén bloqueados y que lleven en la cola más tiempo.

Una forma más natural para manejar esta situación es dividir el estado de No Ejecutando en dos estados, Listo y Bloqueado. Esto se muestra la Figura 3.6. Para gestionarlo correctamente, se han añadido dos estados adicionales que resultarán muy útiles. Estos cinco estados en el nuevo diagrama son los siguientes:

- **Ejecutando.** El proceso está actualmente en ejecución. Para este capítulo asumimos que el computador tiene un único procesador, de forma que sólo un proceso puede estar en este estado en un instante determinado.
- **Listo.** Un proceso que se prepara para ejecutar cuando tenga oportunidad.
- **Bloqueado.** Un proceso que no puede ejecutar hasta que se cumpla un evento determinado o se complete una operación E/S.
- **Nuevo.** Un proceso que se acaba de crear y que aún no ha sido admitido en el grupo de procesos ejecutables por el sistema operativo. Típicamente, se trata de un nuevo proceso que no ha sido cargado en memoria principal, aunque su bloque de control de proceso (BCP) si ha sido creado.
- **Saliente.** Un proceso que ha sido liberado del grupo de procesos ejecutables por el sistema operativo, debido a que ha sido detenido o que ha sido abortado por alguna razón.

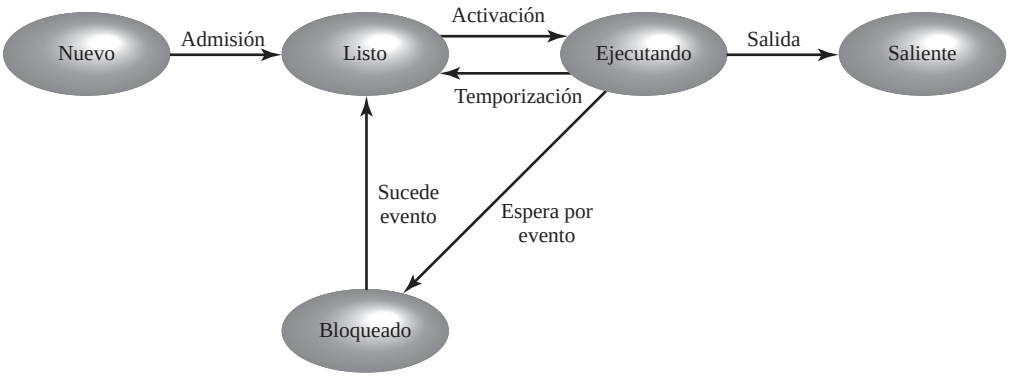


Figura 3.6. Modelo de proceso de cinco estados.

Los estados Nuevo y Saliente son útiles para construir la gestión de procesos. El estado Nuevo se corresponde con un proceso que acaba de ser definido. Por ejemplo, si un nuevo usuario intenta entrar dentro de un sistema de tiempo compartido o cuando se solicita un nuevo trabajo a un sistema de proceso por lotes, el sistema operativo puede definir un nuevo proceso en dos etapas. Primero, el sistema operativo realiza todas las tareas internas que correspondan. Se asocia un identificador a dicho proceso. Se reservan y construyen todas aquellas tablas que se necesiten para gestionar al proceso. En este punto, el proceso se encuentra el estado Nuevo. Esto significa que el sistema operativo ha realizado todas las tareas necesarias para crear el proceso pero el proceso en sí, aún no se ha puesto en ejecución. Por ejemplo, un sistema operativo puede limitar el número de procesos que puede haber en el sistema por razones de rendimiento o limitaciones de memoria principal. Mientras un proceso está en el estado Nuevo, la información relativa al proceso que se necesite por parte del sistema operativo se mantiene en tablas de control de memoria principal. Sin embargo, el proceso en sí mismo no se encuentra en memoria principal. Esto es, el código de programa a ejecutar no se encuentra en memoria principal, y no se ha reservado ningún espacio para los datos asociados al programa. Cuando un proceso se encuentra en el estado Nuevo, el programa permanece en almacenamiento secundario, normalmente en disco[4].

De forma similar, un proceso sale del sistema en dos fases. Primero, el proceso termina cuando alcanza su punto de finalización natural, cuando es abortado debido a un error no recuperable, o cuando otro proceso con autoridad apropiada causa que el proceso se aborte. La terminación mueve el proceso al estado Saliente. En este punto, el proceso no es elegible de nuevo para su ejecución. Las tablas y otra información asociada con el trabajo se encuentran temporalmente preservadas por el sistema operativo, el cual proporciona tiempo para que programas auxiliares o de soporte extraigan la información necesaria. Por ejemplo, un programa de auditoría puede requerir registrar el tiempo de proceso y otros recursos utilizados por este proceso saliente con objeto de realizar una contabilidad de los recursos del sistema. Un programa de utilidad puede requerir extraer información sobre el histórico de los procesos por temas relativos con el rendimiento o análisis de la utilización. Una vez que estos programas han extraído la información necesaria, el sistema operativo no necesita mantener ningún dato relativo al proceso y el proceso se borra del sistema.

La Figura 3.6 indica que tipos de eventos llevan a cada transición de estado para cada proceso; las posibles transiciones son las siguientes:

- **Null → Nuevo.** Se crea un nuevo proceso para ejecutar un programa. Este evento ocurre por cualquiera de las relaciones indicadas en la tabla 3.1.
- **Nuevo → Listo.** El sistema operativo mueve a un proceso del estado nuevo al estado listo cuando éste se encuentre preparado para ejecutar un nuevo proceso. La mayoría de sistemas fijan un límite basado en el número de procesos existentes o la cantidad de memoria virtual que se podrá utilizar por parte de los procesos existentes. Este límite asegura que no haya demasiados procesos activos y que se degrade el rendimiento sistema.
- **Listo → Ejecutando.** Cuando llega el momento de seleccionar un nuevo proceso para ejecutar, el sistema operativo selecciona uno los procesos que se encuentre en el estado Listo. Esta es una tarea la lleva acabo el planificador. El planificador se estudiará más adelante en la Parte Cuatro.
- **Ejecutando → Saliente.** el proceso actual en ejecución se finaliza por parte del sistema operativo tanto si el proceso indica que ha completado su ejecución como si éste se aborta. *Véase* tabla 3.2.

---

[4] En las explicaciones de este párrafo, hemos ignorado el concepto de memoria virtual. En sistemas que soporten la memoria virtual, cuando un proceso se mueve de Nuevo a Listo, su código de programa y sus datos se cargan en memoria virtual. La memoria virtual se ha explicado brevemente en el Capítulo 2 y se examinará con más detalle en el Capítulo 8.

- **Ejecutando → Listo.** La razón más habitual para esta transición es que el proceso en ejecución haya alcanzado el máximo tiempo posible de ejecución de forma ininterrumpida; prácticamente todos los sistemas operativos multiprogramados imponen este tipo de restricción de tiempo. Existen otras posibles causas alternativas para esta transición, que no están incluidas en todos los sistemas operativos. Es de particular importancia el caso en el cual el sistema operativo asigna diferentes niveles de prioridad a diferentes procesos. Supóngase, por ejemplo, que el proceso A está ejecutando a un determinado nivel de prioridad, y el proceso B, a un nivel de prioridad mayor, y que se encuentra bloqueado. Si el sistema operativo se da cuenta de que se produce un evento al cual el proceso B está esperando, moverá el proceso B al estado de Listo. Esto puede interrumpir al proceso A y poner en ejecución al proceso B. Decimos, en este caso, que el sistema operativo ha expulsado al proceso A[5]. Adicionalmente, un proceso puede voluntariamente dejar de utilizar el procesador. Un ejemplo son los procesos que realiza alguna función de auditoría o de mantenimiento de forma periódica.
- **Ejecutando → Bloqueado.** Un proceso se pone en el estado Bloqueado si solicita algo por lo cual debe esperar. Una solicitud al sistema operativo se realiza habitualmente por medio de una llamada al sistema; esto es, una llamada del proceso en ejecución a un procedimiento que es parte del código del sistema operativo. Por ejemplo, un proceso ha solicitado un servicio que el sistema operativo no puede realizar en ese momento. Puede solicitar un recurso, como por ejemplo un fichero o una sección compartida de memoria virtual, que no está inmediatamente disponible. Cuando un proceso quiere iniciar una acción, tal como una operación de E/S, que debe completarse antes de que el proceso continúe. Cuando un proceso se comunica con otro, un proceso puede bloquearse mientras está esperando a que otro proceso le proporcione datos o esperando un mensaje de ese otro proceso.
- **Bloqueado → Listo.** Un proceso en estado Bloqueado se mueve al estado Listo cuando sucede el evento por el cual estaba esperando.
- **Listo → Saliente.** Por claridad, esta transición no se muestra en el diagrama de estados. En algunos sistemas, un padre puede terminar la ejecución de un proceso hijo en cualquier momento. También, si el padre termina, todos los procesos hijos asociados con dicho padre pueden finalizarse.
- **Bloqueado → Saliente.** Se aplican los comentarios indicados en el caso anterior.

Si regresamos a nuestro sencillo ejemplo, Figura 3.7, ahí se muestra la transición entre cada uno de los estados de proceso. La figura 3.8a sugiere la forma de aplicar un esquema de dos colas: la colas de Listos y la cola de Bloqueados. Cada proceso admitido por el sistema, se coloca en la cola de Listos. Cuando llega el momento de que el sistema operativo seleccione otro proceso a ejecutar, selecciona uno de la cola de Listos. En ausencia de un esquema de prioridad, esta cola puede ser una lista de tipo FIFO (*first-in-first-out*). Cuando el proceso en ejecución termina de utilizar el procesador, o bien finaliza o bien se coloca en la cola de Listos o de Bloqueados, dependiendo de las circunstancias. Por último, cuando sucede un evento, cualquier proceso en la cola de Bloqueados que únicamente esté esperando a dicho evento, se mueve a la cola de Listos.

Esta última transición significa que, cuando sucede un evento, el sistema operativo debe recorrer la cola entera de Bloqueados, buscando aquellos procesos que estén esperando por dicho evento. En

---

[5] En general, el término **expulsión** (*preemption*) se define como la reclamación de un recurso por parte de un proceso antes de que el proceso que lo poseía finalice su uso. En este caso, el recurso es el procesador. El proceso está ejecutando y puede continuar su ejecución pero es expulsado por otro proceso que va a entrar a ejecutar.

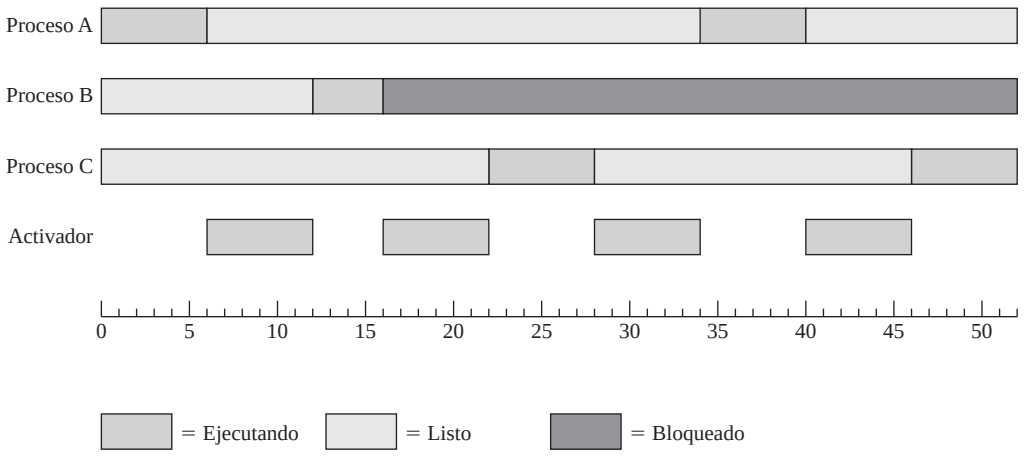


Figura 3.7. Estado de los procesos de la traza de la Figura 3.4.

los sistemas operativos con muchos procesos, esto puede significar cientos o incluso miles de procesos en esta lista, por lo que sería mucho más eficiente tener una cola por cada evento. De esta forma, cuando sucede un evento, la lista entera de procesos de la cola correspondiente se movería al estado de Listo (Figura 3. 8b).

Un refinamiento final sería: si la activación de procesos está dictada por un esquema de prioridades, sería conveniente tener varias colas de procesos listos, una por cada nivel de prioridad. El sistema operativo podría determinar cual es el proceso listo de mayor prioridad simplemente seleccionando éstas en orden.

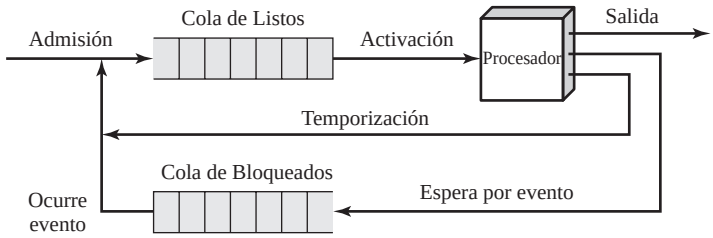
PROCESOS SUSPENDIDOS

La necesidad de intercambio o *swapping*. Los tres principales estados descritos (Listo, Ejecutando, Bloqueado) proporcionan una forma sistemática de modelizar el comportamiento de los procesos y diseñar la implementación del sistema operativo. Se han construido algunos sistemas operativos utilizando únicamente estos tres estados.

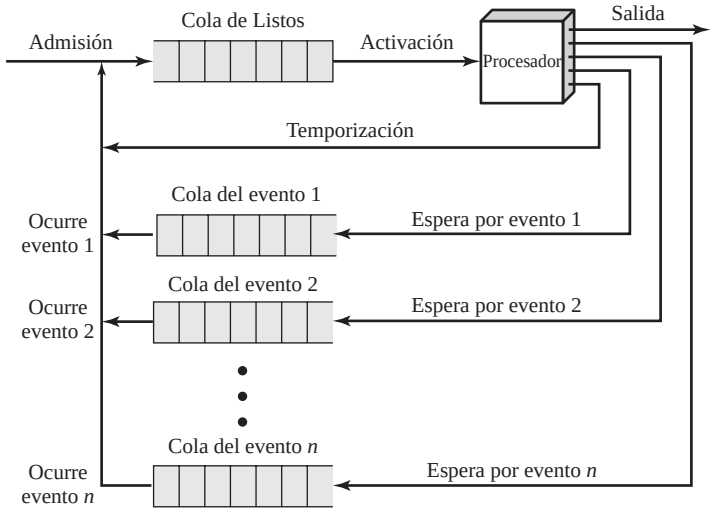
Sin embargo, existe una buena justificación para añadir otros estados al modelo. Para ver este beneficio de nuevos estados, vamos a suponer un sistema que no utiliza memoria virtual. Cada proceso que se ejecuta debe cargarse completamente en memoria principal. Por ejemplo, en la Figura 3.8b, todos los procesos en todas las colas deben residir en memoria principal.

Recuérdese que la razón de toda esta compleja maquinaria es que las operaciones de E/S son mucho más lentas que los procesos de cómputo y, por tanto, el procesador en un sistema monoprogramado estaría ocioso la mayor parte del tiempo. Pero los ajustes de la Figura 3.8b no resuelven completamente el problema. Es verdad que, en este caso, la memoria almacena múltiples procesos y el procesador puede asignarse a otro proceso si el que lo usa se queda bloqueado. La diferencia de velocidad entre el procesador y la E/S es tal que sería muy habitual que todos los procesos en memoria se encontrasen a esperas de dichas operaciones. Por tanto, incluso en un sistema multiprogramado, el procesador puede estar ocioso la mayor parte del tiempo.

¿Qué se puede hacer? La memoria principal puede expandirse para acomodar más procesos. Hay dos fallos en esta solución. Primero, existe un coste asociado a la memoria principal, que, desde un



(a) Cola simple de Bloqueados



(b) Múltiples colas de Bloqueados

Figura 3.8. Modelo de colas de la Figura 3.6.

coste reducido a nivel de bytes, comienza a incrementarse según nos acercamos a un almacenamiento de gigabytes. Segundo, el apetito de los programas a nivel de memoria ha crecido tan rápido como ha bajado el coste de las memorias. De forma que las grandes memorias actuales han llevado a ejecutar procesos de gran tamaño, no más procesos.

Otra solución es el *swapping* (memoria de intercambio), que implica mover parte o todo el proceso de memoria principal al disco. Cuando ninguno de los procesos en memoria principal se encuentra en estado Listo, el sistema operativo intercambia uno de los procesos bloqueados a disco, en la cola de Suspendidos. Esta es una lista de procesos existentes que han sido temporalmente expulsados de la memoria principal, o suspendidos. El sistema operativo trae otro proceso de la cola de Suspendidos o responde a una solicitud de un nuevo proceso. La ejecución continúa con los nuevos procesos que han llegado.

El *swapping*, sin embargo, es una operación de E/S, y por tanto existe el riesgo potencial de hacer que el problema empeore. Pero debido a que la E/S en disco es habitualmente más rápida que la E/S sobre otros sistemas (por ejemplo, comparado con cinta o impresora), el *swapping* habitualmente mejora el rendimiento del sistema.

Con el uso de *swapping* tal y como se ha escrito, debe añadirse un nuevo estado a nuestro modelo de comportamiento de procesos (Figura 3.9a): el estado Suspendido. Cuando todos los procesos en

memoria principal se encuentran en estado Bloqueado, el sistema operativo puede suspender un proceso poniéndolo en el estado Suspendido y transfiriéndolo a disco. El espacio que se libera en memoria principal puede usarse para traer a otro proceso.

Cuando el sistema operativo ha realizado la operación de *swap* (transferencia a disco de un proceso), tiene dos opciones para seleccionar un nuevo proceso para traerlo a memoria principal: puede admitir un nuevo proceso que se haya creado o puede traer un proceso que anteriormente se encontrase en estado de Suspendido. Parece que sería preferible traer un proceso que anteriormente estuviese suspendido, para proporcionar dicho servicio en lugar de incrementar la carga total del sistema.

Pero, esta línea de razonamiento presenta una dificultad: todos los procesos que fueron suspendidos se encontraban previamente en el estado de Bloqueado en el momento de su suspensión. Claramente no sería bueno traer un proceso bloqueado de nuevo a memoria porque podría no encontrarse todavía listo para la ejecución. Se debe reconocer, sin embargo, que todos los procesos en estado Suspendido estaban originalmente en estado Bloqueado, en espera de un evento en particular. Cuando el evento sucede, el proceso no está Bloqueado y está potencialmente disponible para su ejecución.

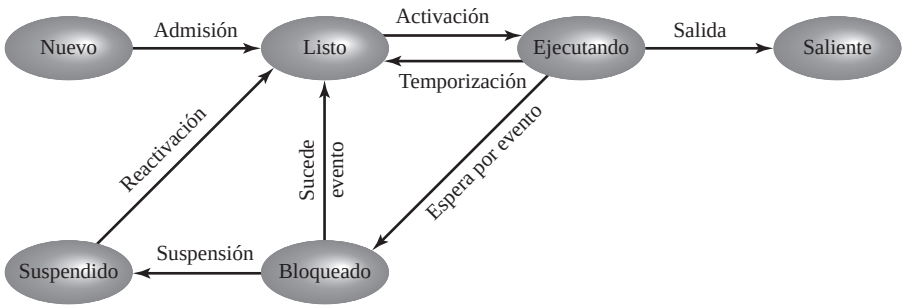
De esta forma, necesitamos replantear este aspecto del diseño. Hay dos conceptos independientes aquí: si un proceso está esperando a un evento (Bloqueado o no) y si un proceso está transferido de memoria a disco (suspendido o no). Para describir estas combinaciones de 2×2 necesitamos cuatro estados:

- **Listo.** El proceso está en memoria principal disponible para su ejecución.
- **Bloqueado.** El proceso está en memoria principal y esperando un evento.
- **Bloqueado/Suspendido.** El proceso está en almacenamiento secundario y esperando un evento.
- **Listo/Suspendido.** El proceso está en almacenamiento secundario pero está disponible para su ejecución tan pronto como sea cargado en memoria principal.

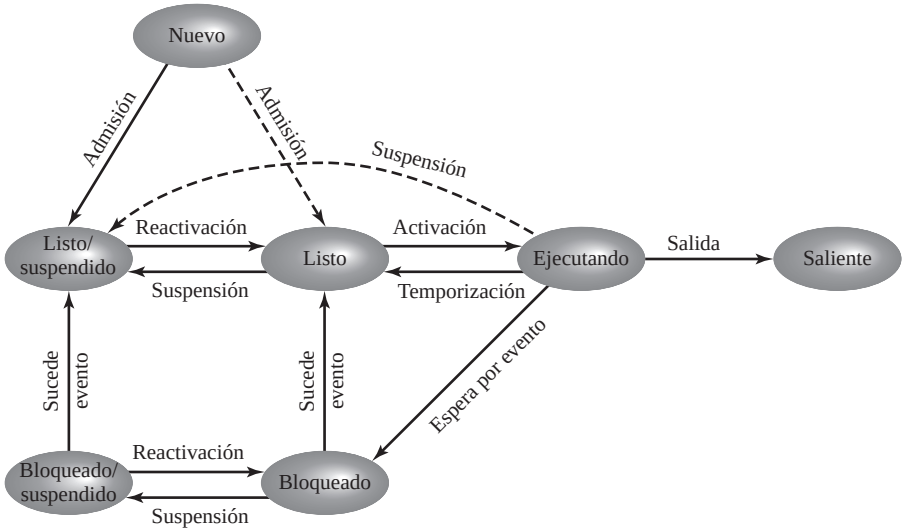
Antes de ir a ver un diagrama de transiciones de estado que incluya estos dos nuevos estados Suspendidos, se debe mencionar otro punto. La explicación ha asumido que no se utiliza memoria virtual y que un proceso está completamente en memoria principal, o completamente fuera de la misma. Con un esquema de memoria virtual, es posible ejecutar un proceso que está sólo parcialmente en memoria principal. Si se hace referencia a una dirección de proceso que no se encuentra en memoria principal, la porción correspondiente del proceso se trae a ella. El uso de la memoria principal podría parecer que elimina la necesidad del *swapping* explícito, porque cualquier dirección necesitada por un proceso que la demande puede traerse o transferirse de disco a memoria por el propio hardware de gestión de memoria del procesador. Sin embargo, como veremos en el Capítulo 8 el rendimiento de los sistemas de memoria virtual pueden colapsarse si hay un número suficientemente grande de procesos activos, todos ellos están parcialmente en memoria principal. De esta forma, incluso con un sistema de memoria virtual, el sistema operativo necesita hacer *swapping* de los procesos de forma explícita y completa, de vez en cuando, con la intención de mejorar el rendimiento.

Veamos ahora, en la Figura 3.9b, el modelo de transición de estados que ha sido diseñado. (Las líneas punteadas de la figura indican transiciones posibles pero no necesarias.) Las nuevas transiciones más importantes son las siguientes:

- **Bloqueado → Bloqueado/Suspendido.** Si no hay procesos listos, entonces al menos uno de los procesos bloqueados se transfiere al disco para hacer espacio para otro proceso que no se encuentra bloqueado. Esta transición puede realizarse incluso si hay procesos listos disponi-



(a) Con un único estado Suspendido



(b) Con dos estados Suspendido

Figura 3.9. Diagrama de transición de estados de procesos con estado suspendidos.

bles, si el sistema operativo determina que el proceso actualmente en ejecución o los procesos listos que desea ejecutar requieren más memoria principal para mantener un rendimiento adecuado.

- **Bloqueado/Suspendido → Listo/Suspendido.** Un proceso en el estado Bloqueado/Suspendido se mueve al estado Listo/Suspendido cuando sucede un evento al que estaba esperando. Nótese que esto requiere que la información de estado concerniente a un proceso suspendido sea accesible para el sistema operativo.

- **Listo/Suspendido → Listo.** Cuando no hay más procesos listos en memoria principal, el sistema operativo necesitará traer uno para continuar la ejecución. Adicionalmente, puede darse el caso de que un proceso en estado Listo/Suspendido tenga mayor prioridad que cualquiera de los procesos en estado Listo. En este caso, el sistema operativo puede haberse diseñado para determinar que es más importante traer un proceso de mayor prioridad que para minimizar el efecto del *swapping*.

- **Listo → Listo/Suspendido.** Normalmente, el sistema operativo preferirá suspender procesos bloqueados que un proceso Listo, porque un proceso Listo se puede ejecutar en ese momento, mientras que un proceso Bloqueado ocupa espacio de memoria y no se puede ejecutar. Sin embargo, puede ser necesario suspender un proceso Listo si con ello se consigue liberar un bloque suficientemente grande de memoria. También el sistema operativo puede decidir suspender un proceso Listo de baja prioridad antes que un proceso Bloqueado de alta prioridad, si se cree que el proceso Bloqueado estará pronto listo.

Otras transiciones interesantes de considerar son las siguientes:

- **Nuevo → Listo/Suspendido y Nuevo a Listo.** Cuando se crea un proceso nuevo, puede añadirse a la cola de Listos o a la cola de Listos/Suspendidos. En cualquier caso, el sistema operativo puede crear un bloque de control de proceso (BCP) y reservar el espacio de direcciones del proceso. Puede ser preferible que el sistema operativo realice estas tareas internas cuanto antes, de forma que pueda disponer de suficiente cantidad de procesos no bloqueados. Sin embargo, con esta estrategia, puede ocurrir que no haya espacio suficiente en memoria principal para el nuevo proceso; de ahí el uso de la transición (Nuevo→Listo/Suspendido). Por otro lado, deseamos hacer hincapié en que la filosofía de creación de procesos diferida, haciéndolo cuanto más tarde, hace posible reducir la sobrecarga del sistema operativo y le permite realizar las tareas de creación de procesos cuando el sistema está lleno de procesos bloqueados.
- **Bloqueado/Suspendido → Bloqueado.** La incursión de esta transición puede parecer propia de un diseño más bien pobre. Después de todo, si hay un proceso que no está listo para ejecutar y que no está en memoria principal, ¿qué sentido tiene traerlo? Pero considérese el siguiente escenario: un proceso termina, liberando alguna memoria principal. Hay un proceso en la cola de Bloqueados/Suspendidos con mayor prioridad que todos los procesos en la cola de Listos/Suspendidos y el sistema operativo tiene motivos para creer que el evento que lo bloquea va a ocurrir en breve. Bajo estas circunstancias, sería razonable traer el proceso Bloqueado a memoria por delante de los procesos Listos.
- **Ejecutando → Listo/Suspendido.** Normalmente, un proceso en ejecución se mueve a la cola de Listos cuando su tiempo de uso del procesador finaliza. Sin embargo, si el sistema operativo expulsa al proceso en ejecución porque un proceso de mayor prioridad en la cola de Bloqueado/Suspendido acaba de desbloquearse, el sistema operativo puede mover al proceso en ejecución directamente a la cola de Listos/Suspendidos y liberar parte de la memoria principal.
- **De cualquier estado → Saliente.** Habitualmente, un proceso termina cuando está ejecutando, bien porque ha completado su ejecución o debido a una condición de fallo. Sin embargo, en algunos sistemas operativos un proceso puede terminarse por el proceso que lo creó o cuando el proceso padre a su vez ha terminado. Si se permite esto, un proceso en cualquier estado puede moverse al estado Saliente.

Otros usos para la suspensión de procesos. Hace poco, hemos definido el concepto de procesos suspendidos como el proceso que no se encuentra en memoria principal. Un proceso que no está en memoria principal no está disponible de forma inmediata para su ejecución, independientemente de si está a la espera o no de un evento.

Podemos analizar el concepto de procesos suspendidos, definiendo un proceso suspendido como el que cumple las siguientes características:

1. El proceso no está inmediatamente disponible para su ejecución.

2. El proceso puede estar o no a la espera de un evento, si es así, la condición de bloqueo es independiente de la condición estar suspendido, y si sucede el evento que lo bloquea, eso no habilita al proceso para su ejecución inmediata.
3. El proceso fue puesto en estado suspendido por un agente: bien el proceso mismo, el proceso padre o el sistema operativo, con el propósito de prevenir su ejecución.
4. El proceso no puede ser recuperado de este estado hasta que el agente explícitamente así lo indique.

Tabla 3.3. Razones para la suspensión de un proceso.

Swapping	El sistema operativo necesita liberar suficiente memoria principal para traer un proceso en estado Listo de ejecución.
Otras razones del sistema operativo	El sistema operativo puede suspender un proceso en segundo plano o de utilidad o un proceso que se sospecha puede causar algún problema.
Solicitud interactiva del usuario	Un usuario puede desear suspender la ejecución de un programa con motivo de su depuración o porque está utilizando un recurso.
Temporización	Un proceso puede ejecutarse periódicamente (por ejemplo, un proceso monitor de estadísticas sobre el sistema) y puede suspenderse mientras espera el siguiente intervalo de ejecución.
Solicitud del proceso padre	Un proceso padre puede querer suspender la ejecución de un descendiente para examinar o modificar dicho proceso suspendido, o para coordinar la actividad de varios procesos descendientes.

La Tabla 3.3 muestra una lista de las condiciones para la suspensión de un proceso. Una de las razones que hemos discutido anteriormente es proporcionar espacio de memoria para traer procesos en estado Listos/Suspendidos o para incrementar el espacio de memoria disponible para los procesos Listos. El sistema operativo puede tener otros motivos para suspender un proceso. Por ejemplo, se puede utilizar un proceso de traza o de auditoría para monitorizar una actividad del sistema; el proceso puede recoger datos a nivel de utilización de varios recursos (procesador, memoria, canales) y la tasa de progreso del uso de los procesos en el sistema. El sistema operativo, bajo el control del operador, puede activar y desactivar este proceso con determinada periodicidad. Si el sistema operativo detecta o sospecha que puede haber un problema, puede suspender procesos. Un ejemplo de esto es un interbloqueo, que ya se discutirá en el Capítulo 6. Otro ejemplo, se detectan problemas en una línea de comunicación, el operador solicita al sistema operativo que suspenda el proceso que utiliza dicha línea hasta realizar algunas pruebas.

Otro tipo de razones están relacionadas con el uso interactivo del sistema. Por ejemplo, si los usuarios sospechan de un programa con algún tipo de *bug*, se puede detener la ejecución de dicho programa, examinando y modificando el programa y sus datos y reanudándolo de nuevo. O puede haber un proceso en segundo plano que recolecta trazas y estadísticas, el cual puede ser activado por parte del usuario.

Algunas consideraciones de tiempo también pueden llevar a activar decisiones de *swapping*. Por ejemplo, si un proceso se activa periódicamente pero está ocioso la mayor parte del tiempo puede ser enviado a disco entre cada una de las ejecuciones que realiza. Un ejemplo es un programa que monitoriza la utilización del sistema y la actividad del usuario.

Por último, un proceso padre puede suspender un proceso descendiente. Por ejemplo, el proceso A lanza al proceso B para realizar una lectura sobre un fichero. Posteriormente, el proceso B encuentra un error en la lectura del fichero y se lo indica al proceso A. El proceso A suspende al proceso B e investiga la causa.

En todos estos casos, la activación de un proceso Suspendido se solicita por medio del agente que inicialmente puso al proceso en suspensión.

3.3. DESCRIPCIÓN DE PROCESOS

El sistema operativo controla los eventos dentro del computador, planifica y activa los procesos para su ejecución por el procesador, reserva recursos para los mismos y responde a las solicitudes de servicios básicos de los procesos de usuario. Fundamentalmente, se piensa en el sistema operativo como en la entidad que gestiona el uso de recursos del sistema por parte de los procesos.

Este concepto se ilustra en la Figura 3.10. En un entorno multiprogramado, hay numerosos procesos (P1,... Pn) creados y residentes en memoria virtual. Cada proceso, durante el transcurso de su ejecución, necesita acceder a ciertos recursos del sistema, incluido el procesador, los dispositivos de E/S y la memoria principal. En la figura, el proceso P1 está ejecutando; al menos parte del proceso está en memoria principal, y controla dos dispositivos de E/S. El proceso P2 está también totalmente en memoria principal pero está bloqueado esperando a disponer de un dispositivo de E/S que está asignado a P1. El proceso Pn se encuentra transferido a disco (*swapping*) y está por tanto suspendido.

Exploraremos los detalles de la gestión de estos recursos por parte del sistema operativo en los próximos capítulos. Aquí nos interesa una cuestión más fundamental: ¿qué información necesita el sistema operativo para controlar los procesos y gestionar los recursos de estos?

ESTRUCTURAS DE CONTROL DEL SISTEMA OPERATIVO

Si el sistema operativo se encarga de la gestión de procesos y recursos, debe disponer de información sobre el estado actual de cada proceso y cada recurso. El mecanismo universal para proporcionar esta información es el siguiente: el sistema operativo construye y mantiene tablas de información sobre cada entidad que gestiona. Se indica una idea general del alcance de esta tarea en la Figura 3.11, que muestra cuatro diferentes tipos de tablas mantenidas por el sistema operativo: memoria, E/S, ficheros y procesos. A pesar de que los detalles difieren de un sistema operativo a otro, fundamentalmente, todos los sistemas operativos mantienen información de estas cuatro categorías.

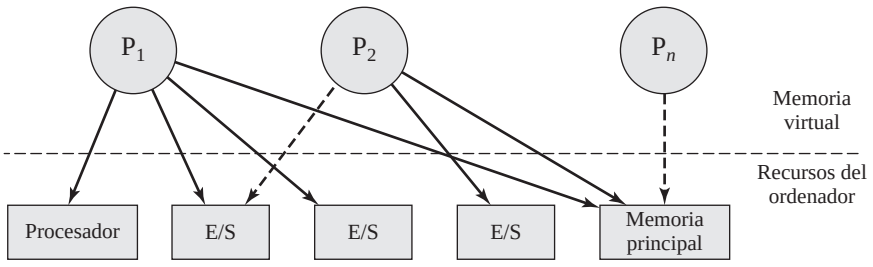


Figura 3.10. Procesos y recursos (reserva de recursos en una instantánea del sistema).

Las **tablas de memoria** se usan para mantener un registro tanto de la memoria principal (real) como de la secundaria (virtual). Parte de la memoria principal está reservada para el uso del sistema operativo; el resto está disponible para el uso de los procesos. Los procesos se mantienen en memoria secundaria utilizando algún tipo de memoria virtual o técnicas de *swapping*. Las tablas de memoria deben incluir la siguiente información:

- Las reservas de memoria principal por parte de los procesos.
- Las reservas de memoria secundaria por parte de los procesos.
- Todos los atributos de protección que restringe el uso de la memoria principal y virtual, de forma que los procesos puedan acceder a ciertas áreas de memoria compartida.
- La información necesaria para manejar la memoria virtual.

Veremos en detalle las estructuras de información para la gestión de memoria en la Parte Tres.

El sistema operativo debe utilizar las **tablas de E/S** para gestionar los dispositivos de E/S y los canales del computador. Pero, en un instante determinado, un dispositivo E/S puede estar disponible o asignado a un proceso en particular. Si la operación de E/S se está realizando, el sistema operativo necesita conocer el estado de la operación y la dirección de memoria principal del área usada como fuente o destino de la transferencia de E/S. La gestión de E/S se examina en detalle en el Capítulo 11.

El sistema operativo también puede mantener las **tablas de ficheros**. Estas tablas proporcionan información sobre la existencia de ficheros, su posición en almacenamiento secundario, su estado ac-

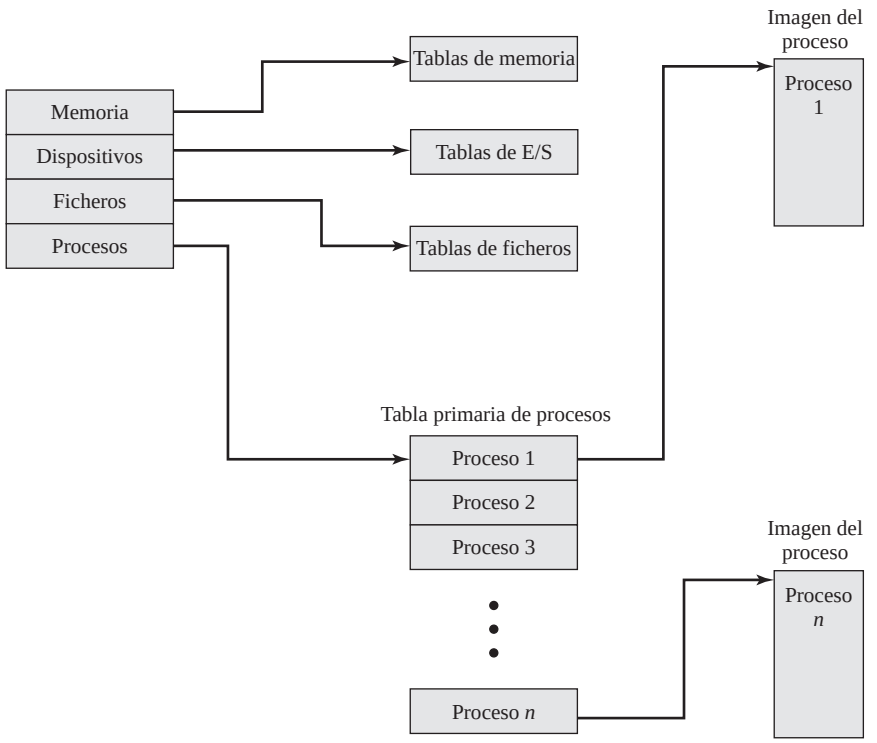


Figura 3.11. Estructura general de las tablas de control del sistema operativo.

tual, y otros atributos. La mayoría de, o prácticamente toda, esta información se puede gestionar por el sistema de ficheros, en cuyo caso el sistema operativo tiene poco o ningún conocimiento sobre los ficheros. En otros sistemas operativos, la gran mayoría del detalle de la gestión de ficheros sí que gestiona en el propio sistema operativo. Este aspecto se explorará en el Capítulo 12.

En último lugar, el sistema operativo debe mantener **tablas de procesos** para gestionar los procesos. La parte restante de esta sección se encuentra dedicada a examinar los requisitos que tienen las tablas de procesos. Antes de proceder con esta explicación, vamos a realizar dos puntualizaciones importantes. Primero, a pesar de que la Figura 3.11 muestra cuatro tipos diferentes de tablas debe quedar claro que estas tablas se encuentran entrelazadas y referenciadas entre sí de alguna manera. Memoria, E/S, y ficheros se gestionan por parte de los procesos, de forma que debe haber algunas referencias en estos recursos, directa o indirectamente, desde las tablas de procesos. Los ficheros indicados en las tablas de ficheros son accesibles mediante dispositivos E/S y estarán, en algún caso, residentes en memoria virtual o principal. Las tablas son también accesibles para el sistema operativo, y además están controladas por la gestión de memoria.

Segundo, ¿cómo puede el sistema operativo crear las tablas iniciales? Ciertamente, el sistema operativo debe tener alguna información sobre el entorno básico, tal como: cuánta memoria física existe, cuáles son los dispositivos de E/S y cuáles son sus identificadores, por ejemplo. Esto es una cuestión de configuración. Esto es, cuando el sistema operativo se inicializa, debe tener acceso a algunos datos de configuración que definen el entorno básico y estos datos se deben crear fuera del sistema operativo, con asistencia humana o por medio de algún software de autoconfiguración.

ESTRUCTURAS DE CONTROL DE PROCESO

Se considerará qué información debe conocer el sistema operativo si quiere manejar y controlar los procesos. Primero, debe conocer dónde están localizados los procesos, y segundo, debe conocer los atributos de los procesos que quiere gestionar (por ejemplo, identificador de proceso y estado del mismo).

Localización de los procesos. Antes de que podamos tratar la cuestión de dónde se encuentran los procesos o cuáles son sus atributos, debemos tratar una cuestión más fundamental: ¿qué es la representación física de un proceso? Como mínimo, un proceso debe incluir un programa o un conjunto de programas a ejecutar. Asociados con estos programas existen unas posiciones de memoria para los datos de variables locales y globales y de cualquier constante definida. Así, un proceso debe consistir en una cantidad suficiente de memoria para almacenar el programa y datos del mismo. Adicionalmente, la ejecución típica de un programa incluye una pila (*véase* Apéndice 1B) que se utiliza para registrar las llamadas a procedimientos y los parámetros pasados entre dichos procedimientos. Por último, cada proceso está asociado a un número de atributos que son utilizados por el sistema operativo para controlar el proceso. Normalmente, el conjunto de estos atributos se denomina **bloque de control del proceso** (BCP)[6]. Nos podemos referir al conjunto de programa, datos, pila, y atributos, como la **imagen del proceso** (Tabla 3.4).

La posición de la imagen del proceso dependerá del esquema de gestión de memoria que se utilice. En el caso más simple, la imagen del proceso se mantiene como un bloque de memoria contiguo, o continuo. Este bloque se mantiene en memoria secundaria, habitualmente disco. Para que el sistema operativo pueda gestionar el proceso, al menos una pequeña porción de su imagen se debe mantener

---

[6] Otros nombres habituales usados para esta estructura de datos son bloque de control de tarea, descriptor del proceso, y descriptor de tarea.

en memoria principal. Para ejecutar el proceso, la imagen del proceso completa se debe cargar en memoria principal o al menos en memoria virtual. Asimismo, el sistema operativo necesita conocer la posición en disco de cada proceso y, para cada proceso que se encuentre en memoria principal, su posición en dicha memoria. Se vio una ligera modificación un poco más compleja de este esquema con el sistema operativo CTSS, en el Capítulo 2. Con CTSS, cuando un proceso se transfiere a disco (*swapping*) parte de la imagen del proceso permanece en memoria principal. De esta forma, el sistema operativo debe mantener en registro qué partes de la imagen del proceso se encuentran todavía en memoria principal.

Tabla 3.4. Elementos típicos de la imagen de un proceso.

Datos del usuario	La parte modificable del espacio de usuario. Puede incluir datos de programa, área de pila de usuario, y programas que puedan ser modificados.
Programa de usuario	El programa a ejecutar
Pila de sistema	Cada proceso tiene una o más pilas de sistema (LIFO) asociadas a él. Una pila se utiliza para almacenar los parámetros y las direcciones de retorno de los procedimientos y llamadas a sistema.
Bloque de control de proceso	Datos necesarios para que el sistema operativo pueda controlar los procesos (*véase* tabla 3.5).

Los sistemas operativos modernos suponen la existencia de un hardware de paginación que permite el uso de la memoria física no contigua, para dar soporte a procesos parcialmente residentes en la memoria principal. En esos casos, una parte de la imagen del proceso se puede encontrar en dicha memoria principal, con las restantes en memoria secundaria[7]. De esta forma, las tablas mantenidas por sistema operativo deben mostrar la localización de cada página de la imagen del proceso.

La Figura 3.11 muestra la estructura de la información de localización de la siguiente forma. Hay una tabla primaria de procesos con una entrada por cada proceso. Cada entrada contiene, al menos, un puntero a la imagen del proceso. Si la imagen del proceso contiene múltiples bloques, esta información se mantiene directamente en la tabla principal del proceso o bien está disponible por referencias cruzadas entre las entradas de las tablas de memoria. Por supuesto, esta es una representación genérica; un sistema operativo en particular puede tener su propia forma de organizar esta información de localización.

Atributos de proceso. Cualquier sistema operativo multiprogramado de la actualidad requiere una gran cantidad de información para manejar cada proceso. Como quedó explicado, esta información se puede considerar que reside en el bloque de control del proceso (BCP). Los diferentes sistemas organizarán esta información de formas diferentes. Al final de este capítulo y en el próximo se muestran varios ejemplos. Por ahora, exploraremos simplemente el tipo de información que el sistema operativo puede utilizar, sin considerar cualquier detalle de cómo esta información está organizada.

---

[7] Esta breve explicación pasa por encima algunos detalles. En particular, en un sistema que utilice memoria virtual, toda imagen de un proceso activo se encuentra siempre en memoria secundaria. Sólo una parte de la imagen se carga en memoria principal, ésta se copia en lugar de moverse. De esta forma, la memoria secundaria mantiene una copia de todos los segmentos y/o páginas. Sin embargo, si la parte de imagen del proceso en memoria principal se ve modificada, la copia en memoria secundaria estará desactualizada hasta que la parte residente en memoria principal se copie de nuevo al disco.

La Tabla 3.5 muestra la lista de los tipos de categorías de información que el sistema operativo requiere para cada proceso. Resulta sorprendente la cantidad de información necesaria para esta gestión. Cuando se vaya apreciando con mayor exactitud las responsabilidades del sistema operativo, esta lista parecerá más razonable.

Podemos agrupar la información del bloque de control del proceso en tres categorías generales:

- Identificación del proceso
- Información de estado del procesador
- Información de control del proceso

Tabla 3.5. Elementos típicos de un bloque de control del proceso (BCP).

Identificación del proceso

Identificadores

Identificadores numéricos que se pueden guardar dentro del bloque de control del proceso:

- identificadores del proceso
- identificador del proceso que creó a este proceso (proceso padre)
- identificador del usuario

Información de estado del procesador

Registros visibles por el usuario

Un registro visible por el usuario es aquel al que se puede hacer referencia por medio del lenguaje máquina que ejecuta el procesador cuando está en modo usuario. Normalmente, existen de 8 a 32 de estos registros, aunque determinadas implementaciones RISC tienen más de 100.

Registros de estado y control

Hay una gran variedad de registros del procesador que se utilizan para el control de operaciones. Estos incluyen:

- *Contador de programa*: contiene la dirección de la siguiente instrucción a ejecutar.
- *Códigos de condición*: resultan de la operación lógica o aritmética más reciente (por ejemplo, signo, cero, acarreo, igual, desbordamiento).
- *Información de estado*: incluyen los *flags* de interrupciones habilitadas/deshabilitadas, modo ejecución.

Puntero de pila

Cada proceso tiene una o más pilas de sistema (LIFO) asociadas a él. Una pila se utiliza para almacenar los parámetros y las direcciones de retorno de los procedimientos y llamadas a sistema. Un puntero de pila apunta a la parte más alta de la pila.

Información de control de proceso

Información de estado y de planificación

Esta es la información que el sistema operativo necesita para analizar las funciones de planificación. Elementos típicos de esta información son:

- *Estado del proceso*: indica si está listo o no el proceso para ser planificado para su ejecución (por ejemplo, Ejecutando, Listo, Esperando, Detenido).
- *Prioridad*: uno o más campos que se pueden utilizar para escribir la prioridad de planificación del proceso. En algunos sistemas, se necesitan múltiples valores (por ejemplo, por-defecto, actual, mayor-disponible).
- *Información relativa a planificación*: esto dependerá del algoritmo de planificación utilizado. Por ejemplo, la cantidad de tiempo que el proceso estaba esperando y la cantidad de tiempo que el proceso ha ejecutado la última vez que estuvo corriendo.
- *Evento*: identificar el evento al cual el proceso está esperando para continuar su ejecución.

Estructuración de datos

Un proceso puede estar enlazado con otro proceso en una cola, anillo o con cualquier otra estructura. Por ejemplo, todos los procesos en estado de espera por un nivel de prioridad en particular pueden estar enlazados en una cola. Un proceso puede mostrar una relación padre-hijo (creador-creado) con otro proceso. El bloque de control del proceso puede contener punteros a otros procesos para dar soporte a estas estructuras.

Comunicación entre procesos

Se pueden asociar diferentes *flags*, señales, y mensajes relativos a la comunicación entre dos procesos independientes. Alguna o toda esta información se puede mantener en el bloque de control de proceso (BCP).

Privilegios de proceso

Los procesos adquieren privilegios de acuerdo con la memoria a la que van a acceder y los tipos de instrucciones que se pueden ejecutar. Adicionalmente, los privilegios se pueden utilizar para usar utilidades de sistema o servicios.

Gestión de memoria

Esta sección puede incluir punteros a tablas de segmentos y/o páginas que describen la memoria virtual asignada a este proceso.

Propia de recursos y utilización

Se deben indicar los recursos controlados por el proceso, por ejemplo, ficheros abiertos. También se puede incluir un histórico de utilización del procesador o de otros recursos; esta información puede ser necesaria para el planificador.

Con respecto al **identificador de proceso**, en prácticamente todos los sistemas operativos, a cada proceso se le asocia un identificador numérico único, que puede ser simplemente un índice en la tabla de procesos principal (Figura 3.11); de otra forma, debe existir una traducción que permita al sistema operativo localizar las tablas apropiadas basándose en dicho identificador de proceso. Este identificador es útil para diferentes cuestiones. Muchas de las otras tablas controladas por el sistema operativo incluyen información que referencia a otro proceso por medio del identificador de proceso. Por ejemplo, se puedan organizar las tablas de memoria para proporcionar un mapa de la memoria principal que indique a qué proceso se tiene asignada cada región. Pueden aparecer referencias similares en las tablas de E/S o de ficheros. Cuando un proceso se comunica con otro proceso, el identificador de proceso informa al sistema operativo del destino de la comunicación. Cuando los procesos pueden crear otros procesos, los identificadores indican el proceso padre y los descendientes en cada caso.

Junto con estos identificadores de proceso, a un proceso se le puede asignar un identificador de usuario que indica qué usuario es responsable del trabajo.

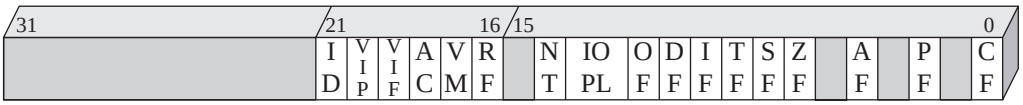
La **información de estado de proceso** indica los contenidos de los registros del procesador. Cuando un proceso está ejecutando, esta información está, por supuesto, en los registros. Cuando un proceso se interrumpe, toda la información de los registros debe salvaguardarse de forma que se pueda restaurar cuando el proceso continúe con su ejecución. La naturaleza y el número de estos registros depende del diseño del procesador. Normalmente, el conjunto de registros incluye registros visibles por usuario, registros de control y de estado, y punteros de pila. Todos estos se describieron en el Capítulo 1.

Se debe notar que, el diseño de todos los procesadores incluye un registro o conjuntos de registros, habitualmente conocidos como palabras de estado de programa (*program status word*, PSW) que contiene información de estado. La PSW típicamente contiene códigos de condición así como otra información de estado. Como ejemplo de una palabra de estado de un procesador se tiene lo que los procesadores Pentium definen como registros EFLAFS (*véase* Figura 3.12 y Tabla 3.6). Esta estructura es utilizada por los sistemas operativos (incluyendo UNIX y Windows) que se ejecuten sobre procesadores de tipo Pentium.

La tercera categoría principal de información del bloque de control de proceso se denomina, a falta de otro nombre mejor, **información de control de proceso**. Esta información adicional la necesita el sistema operativo para controlar y coordinar varios procesos activos. La última parte de la Tabla 3.5 indica el ámbito de esta información. Según examinemos los detalles de funcionamiento del sistema operativo en los sucesivos capítulos, la necesidad de algunos de los elementos de esta lista parecerá más clara.

El contenido de esta figura no debería modificarse. La traducción de los registros de control de una arquitectura en concreto no tiene mucho sentido, puesto que las siglas con las que se denotan están en inglés. El significado de dichas siglas y la traducción del término se encuentran en la siguiente tabla.

La Figura 3.13 sugiere la estructura de las imágenes de los procesos en memoria virtual. Cada imagen de proceso consiste en un bloque de control de proceso, una pila de usuario, un espacio de direcciones privadas del proceso, y cualquier otro espacio de direcciones que el proceso comparta con otros procesos. En la figura, cada imagen de proceso aparece como un rango de direcciones contiguo, pero en una implementación real, este no tiene por qué ser el caso; dependerá del esquema de gestión de memoria y de la forma en la cual se encuentren organizadas las estructuras de control por parte del sistema operativo.



ID = (Indicador de identificación)
VIP = (Interrupción virtual pendiente)
VIF = (Indicador de interrupción virtual)
AC = (Comprobación de alineamiento)
VM = (Modo virtual 8086)
RF = (Indicación de reanudación)
NT = (Indicador de tarea anidada)
IOPL = (Nivel de privilegio de E/S)
OF = (Indicador de desbordamiento)
DF = (Indicador de dirección)
IF = (Indicador de interrupciones habilitadas)
TF = (Indicador de *trap*)
SF = (Indicador de signo)
ZF = (Indicador de cero)
AF = Indicador auxiliar de acarreo)
PF = (Indicador de paridad)
CF = (Indicador de acarreo)

Figura 3.12. Registro EFLAGS de un Pentium II.

Tabla 3.6. Bits del registro EFLAGS de un Pentium.

Bits de control	
AC (*Alignment check*)	Fija si una palabra (o una doble-palabra) se encuentra direccionada en la frontera entre palabras (o dobles-palabras).
ID (*Identification flag*)	Si este bit puede ser puesto a 1 y a 0, este procesador soporta la instrucción CPUID. Esta instrucción proporciona información sobre el vendedor, familia, y modelo.
RF (*Resume flag*)	Permite al programador desactivar las extensiones de depuración de forma que la instrucción pueda volverse a ejecutar después de una excepción de depuración sin causar inmediatamente después otra excepción de depuración.
IOPL (*I/O privilege level*)	Cuando está activado, hace que el procesador genere una excepción para todos los accesos a dispositivo de E/S durante una operación en modo protegido.
DF (*Direction flag*)	Determina cuando una instrucción de procesamiento de cadenas de caracteres incrementa o decrementa los semi-registros de 16 bits SE y DI (para operaciones de 16 bits) o los registros de 32 bits ESE y EDI (para operaciones de 32 bits).
IF (*Interruption enable flag*)	Cuando está puesto a 1, el procesador reconocerá interrupciones externas.
TF (*Trap flag*)	Cuando está puesto a 1, causa una interrupción después de la ejecución de cada instrucción. Su uso está dirigido a la depuración de código.
Bits de modos de operación	
NT (*Nested task flag*)	Indica que la tarea actual está anidada dentro de otra tarea en modo de operación protegido.
VM (*Virtual 8086 mode*)	Permite al programador activar o desactivar el modo virtual 8086, que determina si el procesador funciona como si fuese una máquina 8086.
VIP (*Virtual interrupt pending*)	Usado en el modo virtual 8086 para indicar que una o más interrupciones están a la espera de servicio.
VIF (*Virtual interruption flag*)	Usado en modo virtual 8086 en lugar de IF.
Códigos de condición	
AF (*Auxiliar carry flag*)	Representa el acarreo o resto entre medios bytes de una operación aritmética o lógica de 8 bits usando el registro AL.
CF (*Carry flag*)	Indica el acarreo o el resto por medio del bit situado más a la izquierda después de una operación aritmética. También se modificaron por algunas operaciones de desplazamiento o rotación.
OF (*Overflow flag*)	Indica un desbordamiento (*overflow*) aritmético después de una suma o resta.
PF (*Parity flag*)	Paridad del resultado de una operación aritmética o lógica. 1 indica paridad par; 0 indica paridad impar.
SF (*Sign flag*)	Indica el signo del resultado de una operación aritmética o lógica.
ZF (*Zero flag*)	Indica que el resultado de una operación aritmética o lógica es 0.

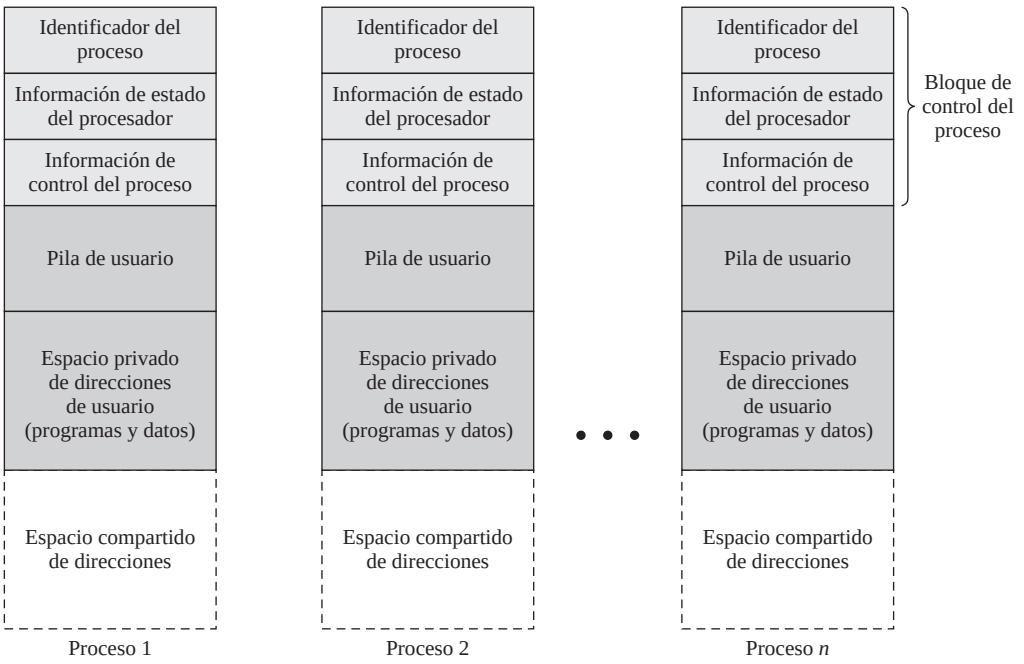


Figura 3.13. Procesos de usuario en memoria virtual.

Como indica la Tabla 3.5, un bloque de control de proceso puede contener información estructural, incluyendo punteros que permiten enlazar bloques de control de proceso entre sí. De esta forma, las colas que se describieron en la sección anterior pueden implementarse como listas enlazadas de bloques de control de proceso. Por ejemplo, la estructura de colas de la Figura 3.8a se puede implementar tal y como sugiere la Figura 3.14.

El papel del bloque de control de proceso. El bloque de control de proceso es la más importante de las estructuras de datos del sistema operativo. Cada bloque de control de proceso contiene toda la información sobre un proceso que necesita el sistema operativo. Los bloques se leen y/o se modifican por la práctica totalidad de los módulos del sistema operativo, incluyendo aquellos relacionados con la planificación, la reserva de recursos, el procesamiento entre regiones, y el análisis y monitorización del rendimiento. Se puede decir que el conjunto de los bloques de control de proceso definen el estado del sistema operativo.

Esto muestra un aspecto importante en el diseño. Un gran número de rutinas dentro del sistema operativo necesitarán acceder a información de los bloques de control de proceso. Proporcionar acceso directo a estas tablas no es difícil. Cada proceso lleva asociado un único identificador, que puede utilizarse para indexar dentro de una tabla de punteros a bloques de control de proceso. La dificultad no reside en el acceso sino en la protección. Dos posibles problemas se pueden presentar son:

- Un fallo en una simple rutina, como un manejador de interrupción, puede dañar los bloques de control de proceso, que puede destruir la capacidad del sistema para manejar los procesos afectados.
- Un cambio en la estructura o en la semántica de los bloques de control de procesos puede afectar a un gran número de módulos del sistema operativo.

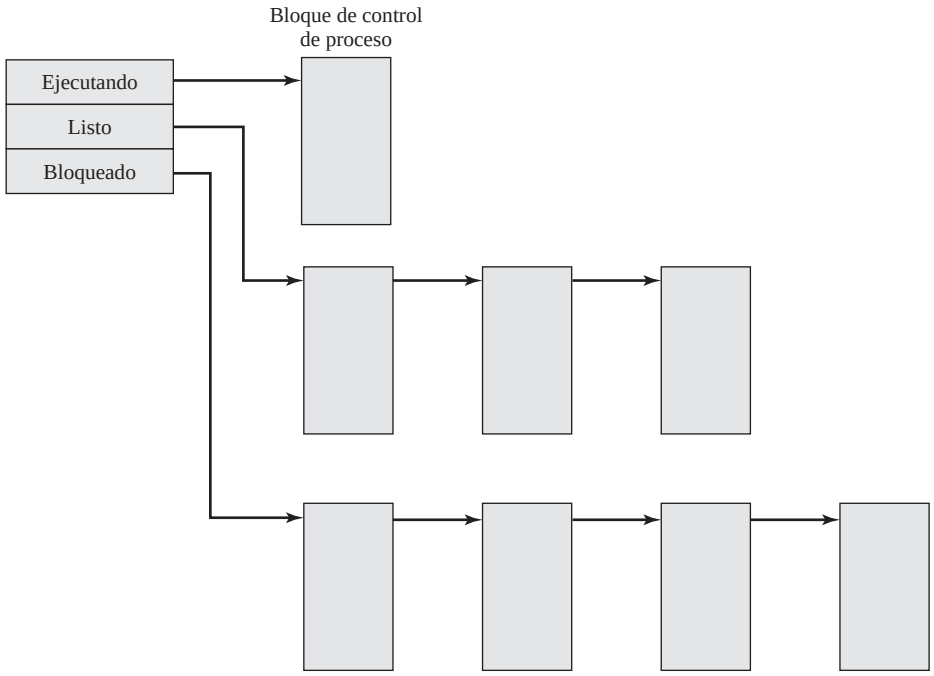


Figura 3.14. Estructuras de listas de procesos.

Estos problemas se pueden ser tratar obligando a que todas rutinas del sistema operativo pasen a través de una rutina de manejador, cuyo único trabajo es proteger a los bloques de control de proceso, y que es la única que realiza el arbitraje en las operaciones de lectura y escritura de dichos bloques. Los factores a equilibrar en el uso de está rutina son, por un lado los aspectos de rendimiento, y por el otro, el grado en el cual el resto del software del sistema puede verificarse para determinar su corrección.

3.4. CONTROL DE PROCESOS

MODOS DE EJECUCIÓN

Antes de continuar con nuestra explicación sobre cómo el sistema operativo gestiona los procesos, necesitamos distinguir entre los modos de ejecución del procesador, normalmente asociados con el sistema operativo y los asociados con los programas de usuario. Muchos procesadores proporcionan al menos dos modos de ejecución. Ciertas instrucciones se pueden ejecutar en modos privilegiados únicamente. Éstas incluirían lectura y modificación de los registros de control, por ejemplo la palabra de estado de programa; instrucciones de E/S primitivas; e instrucciones relacionadas con la gestión de memoria. Adicionalmente, ciertas regiones de memoria sólo se pueden acceder en los modos más privilegiados.

El modo menos privilegiado a menudo se denomina **modo usuario**, porque los programas de usuario típicamente se ejecutan en este modo. El modo más privilegiado se denomina **modo sistema**, **modo control** o **modo núcleo**. Este último término se refiere al núcleo del sistema operativo, que es la parte del sistema operativo que engloba las funciones más importantes del sistema. La Tabla 3.7 lista las funciones que normalmente se encuentran dentro del núcleo del sistema operativo.

Tabla 3.7. Funciones típicas de un núcleo de sistema operativo.

Gestión de procesos

- Creación y terminación de procesos
- Planificación y activación de procesos
- Intercambio de procesos
- Sincronización de procesos y soporte para comunicación entre procesos
- Gestión de los bloques de control de proceso

Gestión memoria

- Reserva de espacio direcciones para los procesos
- *Swapping*
- Gestión de páginas y segmentos

Gestión E/S

- Gestión de *buffers*
- Reserva de canales de E/S y de dispositivos para los procesos

Funciones de soporte

- Gestión de interrupciones
- Auditoría
- Monitorización

El motivo por el cual se usan los otros modos es claro. Se necesita proteger al sistema operativo y a las tablas clave del sistema, por ejemplo los bloques de control de proceso, de la interferencia con programas de usuario. En modo núcleo, el software tiene control completo del procesador y de sus instrucciones, registros, y memoria. Este nivel de control no es necesario y por seguridad no es recomendable para los programas de usuario.

Aparecen dos cuestiones: ¿cómo conoce el procesador en que modo está ejecutando y cómo este modo puede modificarse? En lo referente la primera cuestión, existe típicamente un bit en la palabra de estado de programa (PSW) que indica el modo de ejecución. Este bit se cambia como respuesta a determinados eventos. Habitualmente, cuando un usuario realiza una llamada a un servicio del sistema operativo o cuando una interrupción dispara la ejecución de una rutina del sistema operativo, este modo de ejecución se cambia a modo núcleo y; tras la finalización del servicio, el modo se fija de nuevo a modo usuario. Por ejemplo, si consideramos el procesador Intel Itanium, que implementa la arquitectura IA-64 de 64 bits, el procesador tiene un registro de estado (psr) que incluye un campo de los 2 bits llamado cpl (*current privilege level*). El Nivel 0 es en el nivel con más privilegios mientras que el Nivel 3 es el nivel con menos privilegios. La mayoría de los sistemas operativos, como Linux, utilizar el Nivel 0 para el núcleo y uno de los otros niveles como nivel de usuario. Cuando llega una interrupción, el procesador limpia la mayor parte de los bits en psr, incluyendo el campo cpl. Esto automáticamente pone en 0 el cpl. Al final de la rutina de manejo de interrupción, la última instrucción que se ejecuta es irt (interrupt return). Esta instrucción hace que el procesador restaure el psr del programa interrumpido, que restaura a su vez el nivel de privilegios de dicho programa. Una secuencia similar ocurre cuando la aplicación solicita una llamada al sistema. Para los Itanium, una aplicación solicita la llamada sistema indicando un identificador de llamada de sistema

y los argumentos de dicha llamada en unas áreas definidas y posteriormente ejecutando una instrucción especial que se puede invocar en modo usuario y que tiene como objeto transferir el control al núcleo.

CREACIÓN DE PROCESOS

En la Sección 3.1 hemos comentado los eventos que llevará creación de uno proceso. Una vez vistas las estructuras de datos asociadas a un proceso, ahora posiciones está en posición de describir brevemente los pasos que llevan a la verdadera creación de un proceso.

Una vez que el sistema operativo decide, por cualquier motivo (Tabla 3.1), crear un proceso procederá de la siguiente manera:

1. **Asignar un identificador de proceso único al proceso**. En este instante, se añade una nueva entrada a la tabla primaria de procesos, que contiene una entrada por proceso.
2. **Reservar espacio para proceso**. Esto incluye todos los elementos de la imagen del proceso. Para ello, el sistema operativo debe conocer cuánta memoria se requiere para el espacio de direcciones privado (programas y datos) y para la pila de usuario. Estos valores se pueden asignar por defecto basándonos en el tipo de proceso, o pueden fijarse en base a la solicitud de creación del trabajo remitido por el usuario. Si un proceso es creado por otro proceso, el proceso padre puede pasar los parámetros requeridos por el sistema operativo como parte de la solicitud de la creación de proceso. Si existe una parte del espacio direcciones compartido por este nuevo proceso, se fijan los enlaces apropiados. Por último, se debe reservar el espacio para el bloque de control de proceso (BCP).
3. **Inicialización del bloque de control de proceso**. La parte de identificación de proceso del BCP contiene el identificador del proceso así como otros posibles identificadores, tal como el indicador del proceso padre. En la información de estado de proceso del BCP, habitualmente se inicializa con la mayoría de entradas a 0, excepto el contador de programa (fijado en el punto entrada del programa) y los punteros de pila de sistema (fijados para definir los límites de la pila del proceso). La parte de información de control de procesos se inicializa en base a los valores por omisión, considerando también los atributos que han sido solicitados para este proceso. Por ejemplo, el estado del proceso se puede inicializar normalmente a Listo o Listo/Suspendido. La prioridad se puede fijar, por defecto, a la prioridad más baja, a menos que una solicitud explicita la eleve a una prioridad mayor. Inicialmente, el proceso no debe poseer ningún recurso (dispositivos de E/S, ficheros) a menos que exista una indicación explícita de ello o que haya sido heredados del padre.
4. **Establecer los enlaces apropiados**. Por ejemplo, si el sistema operativo mantiene cada cola del planificador como una lista enlazada, el nuevo proceso debe situarse en la cola de Listos o en la cola de Listos/Suspendidos.
5. **Creación o expansión de otras estructuras de datos**. Por ejemplo, el sistema operativo puede mantener un registro de auditoría por cada proceso que se puede utilizar posteriormente a efectos de facturación y/o de análisis de rendimiento del sistema.

CAMBIO DE PROCESO

A primera vista, la operación de cambio de proceso puede parecer sencilla. En algunos casos, un proceso en ejecución se interrumpe para que el sistema operativo asigne a otro proceso el estado de Ejecutando y de esta forma establecer el turno entre los procesos. Sin embargo, es necesario tener en

cuenta algunas consideraciones de diseño. Primero, ¿qué evento dispara los cambios de proceso? Otra consideración que se debe tener en cuenta es la distinción entre cambio de proceso (*process switching*) y cambio de modo (*mode switching*). Por último, ¿qué debe hacer el sistema operativo con las diferentes estructuras que gestiona para proporcionar un cambio de proceso?

Cuándo se realiza el cambio de proceso. Un cambio de proceso puede ocurrir en cualquier instante en el que el sistema operativo obtiene el control sobre el proceso actualmente en ejecución. La Tabla 3.8 indica los posibles momentos en los que el sistema operativo puede tomar dicho control.

Primero, consideremos las interrupciones del sistema. Realmente, podemos distinguir, como hacen muchos sistemas, dos tipos diferentes de interrupciones de sistema, unas simplemente denominadas interrupciones, y las otras denominadas *traps*. Las primeras se producen por causa de algún tipo de evento que es externo e independiente al proceso actualmente en ejecución, por ejemplo la finalización de la operación de E/S. Las otras están asociadas a una condición de error o excepción generada dentro del proceso que está ejecutando, como un intento de acceso no permitido a un fichero. Dentro de una **interrupción** ordinaria, el control se transfiere inicialmente al manejador de interrupción, que realiza determinadas tareas internas y que posteriormente salta a una rutina del sistema operativo, encargada de cada uno de los tipos de interrupciones en particular. Algunos ejemplos son:

- **Interrupción de reloj.** El sistema operativo determinar si el proceso en ejecución ha excedido o no la unidad máxima de tiempo de ejecución, denominada **rodaja de tiempo** (*time slice*). Esto es, una rodaja de tiempo es la máxima cantidad de tiempo que un proceso puede ejecutar antes de ser interrumpido. En dicho caso, este proceso se puede pasar al estado de Listo y se puede activar otro proceso.
- **Interrupción de E/S.** El sistema operativo determina qué acción de E/S ha ocurrido. Si la acción de E/S constituye un evento por el cual están esperando uno o más procesos, el sistema operativo mueve todos los procesos correspondientes al estado de Listos (y los procesos en estado Bloqueado/Suspendido al estado Listo/Suspendido). El sistema operativo puede decidir si reanuda la ejecución del proceso actualmente en estado Ejecutando o si lo expulsa para proceder con la ejecución de un proceso Listo de mayor prioridad.

Tabla 3.8. Mecanismos para la interrupción de la ejecución de un proceso.

Mecanismo	Causa	Uso
Interrupción	Externa a la ejecución del proceso actualmente en ejecución.	Reacción ante un evento externo asíncrono.
Trap	Asociada a la ejecución de la instrucción actual.	Manejo de una condición de error o de excepción.
Llamada al sistema	Solicitud explícita.	Llamada a una función del sistema operativo.

- **Fallo de memoria.** El procesador se encuentra con una referencia a una dirección de memoria virtual, a una palabra que no se encuentra en memoria principal. El sistema operativo debe traer el bloque (página o segmento) que contiene la referencia desde memoria secundaria a memoria principal. Después de que se solicita la operación de E/S para traer el bloque a memoria, el proceso que causó el fallo de memoria se pasa al estado de Bloqueado; el sistema operativo realiza un cambio de proceso y pone a ejecutar a otro proceso. Después de que el bloque de memoria solicitado se haya traído, el proceso pasará al estado Listo.

Con un ***trap***, el sistema operativo conoce si una condición de error o de excepción es irreversible. Si es así, el proceso en ejecución se pone en el estado Saliente y se hace un cambio de proceso. De no ser así, el sistema operativo actuará dependiendo de la naturaleza del error y su propio diseño. Se puede intentar un procedimiento de recuperación o simplemente la notificación al usuario, pudiendo implicar tanto un cambio de proceso como la continuación de la ejecución del proceso actual.

Por último, el sistema operativo se puede activar por medio de una **llamada al sistema** procedente del programa en ejecución. Por ejemplo, si se está ejecutando un proceso y se ejecuta una operación que implica una llamada de E/S, como abrir un archivo. Esta llamada implica un salto a una rutina que es parte del código del sistema operativo. La realización de una llamada al sistema puede implicar en algunos casos que el proceso que la realiza pase al estado de Bloqueado.

Cambio de modo. En el Capítulo 1, discutimos la inclusión de una fase de interrupción como parte del ciclo de una instrucción. Recordando esto, en la fase de interrupción, el procesador comprueba que no exista ninguna interrupción pendiente, indicada por la presencia de una señal de interrupción. Si no hay interrupciones pendientes, el procesador pasa a la fase de búsqueda de instrucción, siguiendo con el programa del proceso actual. Si hay una interrupción pendiente, el proceso actúa de la siguiente manera:

1. Coloca el contador de programa en la dirección de comienzo de la rutina del programa manejador de la interrupción.
2. Cambia de modo usuario a modo núcleo de forma que el código de tratamiento de la interrupción pueda incluir instrucciones privilegiadas.

El procesador, acto seguido, pasa a la fase de búsqueda de instrucción y busca la primera instrucción del programa de manejo de interrupción, que dará servicio a la misma. En este punto, habitualmente, el contexto del proceso que se ha interrumpido se salvaguarda en el bloque de control de proceso del programa interrumpido.

Una pregunta que se puede plantear es, ¿qué constituye el contexto que se debe salvaguardar? La respuesta es que se trata de toda la información que se puede ver alterada por la ejecución de la rutina de interrupción y que se necesitará para la continuación del proceso que ha sido interrumpido. De esta forma, se debe guardar la parte del bloque de control del proceso que hace referencia a la información de estado del procesador. Esto incluye el contador de programa, otros registros del procesador, y la información de la pila.

¿Se necesita hacer algo más? Eso depende de qué ocurra luego. El manejador de interrupción es habitualmente un pequeño programa que realiza unas pocas tareas básicas relativas a la interrupción. Por ejemplo, borra el *flag* o indicador que señala la presencia de interrupciones. Puede enviar una confirmación a la entidad que lanzó dicha interrupción, como por ejemplo el módulo de E/S. Y puede realizar algunas tareas internas variadas relativas a los efectos del evento que causó la interrupción. Por ejemplo, si la interrupción se refiere a un evento de E/S, el manejador de interrupción comprobará la existencia o no de una condición de error. Si ha ocurrido un error, el manejador mandará una señal al proceso que solicitó dicha operación de E/S. Si la interrupción proviene del reloj, el manejador la va a pasar el control al activador, el cual decidirá pasar a otro proceso debido a que la rodaja de tiempo asignada a ese proceso ha expirado.

¿Qué pasa con el resto de información del bloque de control de proceso? Si a esta interrupción le sigue un cambio de proceso a otro proceso, se necesita hacer algunas cosas más. Sin embargo, en muchos sistemas operativos, la existencia de una interrupción no implica necesariamente un cambio de proceso. Es posible, por tanto, que después de la ejecución de la rutina de interrupción, la ejecución se reanude con el mismo proceso. En esos casos sólo se necesita salvaguardar la información del estado del procesador cuando se produce la interrupción y restablecerlo cuando se reanude la

ejecución del proceso. Habitualmente, las operaciones de salvaguarda y recuperación se realizan por hardware.

Cambio del estado del proceso. Esta claro, por tanto, que el cambio de modo es un concepto diferente del cambio de proceso[8]. Un cambio de modo puede ocurrir sin que se cambie el estado del proceso actualmente en estado Ejecutando. En dicho caso, la salvaguarda del estado y su posterior restauración comportan sólo una ligera sobrecarga. Sin embargo, si el proceso actualmente en estado Ejecutando, se va a mover a cualquier otro estado (Listo, Bloqueado, etc.), entonces el sistema operativo debe realizar cambios sustanciales en su entorno. Los pasos que se realizan para un cambio de proceso completo son:

1. Salvar el estado del procesador, incluyendo el contador de programa y otros registros.
2. Actualizar el bloque de control del proceso que está actualmente en el estado Ejecutando. Esto incluye cambiar el estado del proceso a uno de los otros estados (Listo, Bloqueado, Listo/Suspendido, o Saliente). También se tienen que actualizar otros campos importantes, incluyendo la razón por la cual el proceso ha dejado el estado de Ejecutando y demás información de auditoría.
3. Mover el bloque de control de proceso a la cola apropiada (Listo, Bloqueado en el evento *i*, Listo/Suspendido).
4. Selección de un nuevo proceso a ejecutar; esta cuestión se analiza con más detalle en la Parte Cuatro.
5. Actualizar el bloque de control del proceso elegido. Esto incluye pasarlo al estado Ejecutando.
6. Actualizar las estructuras de datos de gestión de memoria. Esto se puede necesitar, dependiendo de cómo se haga la traducción de direcciones; estos aspectos se cubrirán en la Parte Tres.
7. Restaurar el estado del procesador al que tenía en el momento en el que el proceso seleccionado salió del estado Ejecutando por última vez, leyendo los valores anteriores de contador de programa y registros.

Por tanto, el cambio de proceso, que implica un cambio en el estado, requiere un mayor esfuerzo que un cambio de modo.

EJECUCIÓN DEL SISTEMA OPERATIVO

En el Capítulo 2, señalamos dos aspectos intrínsecos del sistema operativo:

- El sistema operativo funciona de la misma forma que cualquier software, en el sentido que un sistema operativo es un conjunto de programas ejecutados por un procesador.
- El sistema operativo, con frecuencia, cede el control y depende del procesador para recuperar dicho control.

---

[8] El término **cambio de contexto** (*context switch*) se usa en literatura sobre sistemas operativos y en libros de texto. Desafortunadamente, aunque la mayoría de estas referencias utilizan este término para lo que en este libro se ha denominado cambio de proceso, otras fuentes lo usan para referirse al cambio de modo o incluso al cambio de thread (definido en el siguiente capítulo). Para evitar ambigüedades, este término no se ha utilizado en este libro.